



**University of
Zurich**^{UZH}

Enhancing the Scalability of a Sharded Blockchain

*Raphael Beckmann
Uetikon, Zürich, Switzerland
Student ID: 16-711-467*

Supervisor: Sina Rafati, Dr. Thomas Bocek
Date of Submission: March 25, 2020

Abstract

Deutsch

Um neue Blockchaintechnologien testen zu können, wurde die BAZO Forschungsblockchain von der Universität Zürich und der Hochschule für Technik Rapperswil entworfen und entwickelt. Es wurden bereits mehrere Projekte mit BAZO durchgeführt, welche Funktionalitäten wie beispielsweise einen Proof-Of-Stake Mechanismus hinzufügten. Die neusten Beiträge sind aber die Sharding und Transaction Aggregation Mechanismen. Der erste Beitrag dieser Bachelorarbeit war das Zusammenfügen der zwei zuvor erwähnten Projekte, was in Folge zu einer aktuellen Version von BAZO mit sämtlichen relevanten Technologien, die dafür entwickelt worden sind, führte. Anschliessend wurden verschiedene Ansätze um die Skalierbarkeit von BAZO weiter zu erhöhen geprüft und implementiert. Diese Beiträge beinhalten beispielsweise einen Mechanismus, um mehr Miner zu einer Shard hinzuzufügen sowie Mechanismen zur Beschleunigung der Datenbank. Schlussendlich wurde ein neuer Konsensusmechanismus, welcher von mehreren modernen Blockchainprojekten der letzten Jahre inspiriert wurde, implementiert, um die Sicherheits- und Skalierbarkeitsprobleme von BAZO möglichst effizient zu lösen. Diese finale Implementierung wurde gründlich getestet und rigoros dokumentiert. Am Ende der Arbeit werden zudem Richtlinien für die Weiterentwicklung von BAZO aufgeführt.

English

In order to test new blockchain technologies, the BAZO research blockchain was designed and programmed by the University of Zürich and the University of Applied Sciences Rapperswil. There have already been numerous iterations of development on BAZO, adding features such as a Proof-Of-Stake mechanism. The most recent additions, however, were Sharding and Transaction Aggregation. The first contribution of this thesis is the merging of the two previously mentioned projects, creating an up-to-date version of the relevant technologies which have been realized on BAZO. Moreover, various ways on how to further increase BAZO's scalability were designed and implemented in the next phase. Those improvements include, among others, adding more miners to a shard, as well as database accelerations. Finally, in order to overcome the serious security and scalability issues that BAZO suffered at the moment of writing the thesis, a new consensus mechanism was implemented, which was inspired by several cutting-edge blockchain technologies in recent years. This final implementation was then thoroughly tested and rigorously documented. At the end, there are some guidelines for the further development of BAZO.

Acknowledgments

First of all, I would like to thank my supervisors Sina Rafati and Dr. Thomas Bocek for their continuous assistance. In the countless meetings, they helped me to find solutions to my problems and gave valuable suggestions for new approaches.

Furthermore, I would like to thank Fabio Maddaloni and Kürsat Aydinli for patiently responding to my questions regarding the BAZO codebase and their specific implementations.

Finally, I would also like to thank Professor Burkhard Stiller for the opportunity to perform research in this exciting field with the Communication Systems Group at the University of Zürich.

Contents

Abstract	i
Acknowledgments	iii
1 Introduction	1
1.1 Motivation	1
1.2 BAZO	1
1.3 Description of Work	2
1.4 Thesis Outline	2
2 Related Work	3
2.1 Transaction Aggregation	3
2.1.1 Motivation	3
2.1.2 Design & Implementation	4
2.2 Sharding	4
2.2.1 Motivation	4
2.2.2 Design & Implementation	5
2.3 Further Literature	5
2.3.1 Zilliqa	6
2.3.2 RapidChain	6
2.3.3 QuarkChain	7
2.3.4 OmniLedger	7
2.3.5 Libra	8
2.3.6 Literature Summary	8

3	Combining Transaction Aggregation & Sharding	11
3.1	Overview	11
3.2	Solved Issues & Bugs	12
3.2.1	Decreasing the Complexity	12
3.2.2	Different Protocol	12
3.2.3	General Sharding Bugfixes	13
3.2.4	Edge Cases in Sharding	13
3.2.5	Epoch Block Finality	15
4	Scalability Improvements	17
4.1	Assessing the State of the Merged Version	17
4.1.1	Reaching a Full Mesh Network	17
4.1.2	More Miners per Shard	18
4.1.3	Network Improvements	24
4.1.4	Testbed Implementation	26
4.1.5	Metrics Used	28
4.1.6	Testing Results & Evaluation	29
4.1.7	Testing Conclusions	33
4.2	Attempts to Increase the Scalability	33
4.2.1	Database Improvements	33
4.2.2	Enhancing the Synchronization Mechanism	35
4.2.3	Evaluation of the Improvements	36
4.2.4	Testing Conclusions	38
4.3	Conclusions from the Scalability Improvement	38

5	Committee Design	39
5.1	Motivation	39
5.2	Assumptions	40
5.3	Entity Management & Assignment	41
5.4	Data Transactions	43
5.5	Transaction Aggregation	44
5.6	Transaction Management & Assignment	44
5.7	Communication & Synchronization	45
5.7.1	Motivation	46
5.7.2	Inter-Shard Communication & Synchronization	46
5.7.3	Inter-Committee Communication & Synchronization	46
5.7.4	Validator-Committee Communication & Synchronization	47
5.8	Authentication & Validation	48
5.8.1	Authentication Strategy	49
5.8.2	Validation Strategy	49
5.9	Assessment of the Design	50
5.9.1	Architecture	50
5.9.2	Use Cases	51
5.9.3	Permissions & Coverage	52
5.9.4	Scalability	53
6	Implementation	55
6.1	Consensus Mechanism	55
6.1.1	Consensus in the Shards	55
6.1.2	Consensus in the Committee	57
6.1.3	Global Consensus	61
6.2	Transaction Life Cycle	61
6.2.1	Sending Transactions into the Network	61

6.2.2	Transaction Assignment to the Shards	62
6.2.3	Shard Mining	63
6.2.4	Committee Validation	64
6.3	Data Management & Data Retrieval	64
6.3.1	Data Summary	64
6.3.2	Storing Mechanism	65
6.4	Authentication & Validation	66
6.4.1	Signing a Message	66
6.4.2	Verification of a Signature	67
6.4.3	Validation of a Message	69
6.4.4	Security Issues in the Sharded Version of BAZO	71
6.5	New State Design	72
6.6	New Communication Mechanisms	72
6.6.1	State Transition	73
6.6.2	Transaction Assignment	73
6.6.3	Committee Check	73
6.7	New Transactions	74
6.7.1	CommitteeTx	74
6.7.2	DataTx	74
6.7.3	AggDataTx	75
6.7.4	FineTx	75
6.8	Timestamps	76
7	Evaluation	77
7.1	Theoretical Performance Evaluation	77
7.2	Local Testbed	77
7.2.1	Implementation	77
7.2.2	One Committee Member	78

<i>CONTENTS</i>	ix
7.2.3 Two Committee Members	80
7.2.4 Three Committee Members	81
7.2.5 Four Committee Members	82
7.2.6 Testing Conclusions	84
7.3 Online Testbed	85
7.3.1 Implementation	85
7.3.2 Current Problems	85
7.4 Future Work	86
8 Summary & Conclusions	87
Bibliography	89
Abbreviations	93
List of Figures	93
List of Tables	95
A Installation Guidelines	99
A.1 Installation	99
A.2 Online Testbed	100
B Contents of the CD	101

Chapter 1

Introduction

In today's world, IoT devices are becoming more and more popular. According to an estimation by Ericsson, there will be around 18 billion connected devices which are related to IoT by 2022 [2]. In order to process and store the data that they produce in a fast and reliable way, blockchain technology can be used because it uses consensus mechanisms to validate transactions and store them inside a block [3]. However, traditional blockchains such as Bitcoin and Ethereum have notoriously low transaction validation rates [25], which hinders them from leveraging the full potential that the technology could have.

1.1 Motivation

The BAZO blockchain was developed to find mechanisms to overcome the scalability issues of current blockchain projects. Significant progress has been made, especially with the implementation of *Transaction Aggregation* [27] and *Sharding* [11]. However, network instabilities, along with other minor issues remain and the scalability has to be further improved in order to meet the growing requirements of the industry and research applications. In fact, when talking about blockchains, performance is always a major aspect. The reason for this is that if the blockchain can't keep up with its workload, the resulting congestion can lead to hard forks which unfortunately almost became an industry standard nowadays [22], which is something to avoid. Regardless of the application, blockchains need to provide the required performance such that businesses are willing to adopt the technology to their operations [22].

1.2 BAZO

BAZO is a research blockchain mostly developed by the University of Zürich in 2017 [30]. At the time when this thesis was being completed, it implemented a Proof of Stake consensus mechanism [12]. A lot of work has been conducted to extend the functionalities

of the blockchain, with the most recent ones being *Transaction Aggregation* [27] and *Sharding* [11].

As it has been shown in the past, BAZO can be adapted to serve a multiplicity of use cases and scenarios. In recent time, most of the efforts, however, have been concentrated on increasing the scalability with regards to memory efficiency and transactions validated per second. The goal of this thesis therefore was to contribute to this effort and increase the scalability of BAZO. For this work, it was aimed to reach a transaction validation rate of 1000-2000 transactions per second by increasing the number of miners in the blockchain, which is about 20 times faster than the previously fastest implementation of BAZO [11].

1.3 Description of Work

The purpose of this thesis was to first consolidate the recent efforts to increase the scalability of BAZO to a unified system. Those efforts include the *Transaction Aggregation* mechanism as well as the *Sharding* mechanism which were both developed at the University of Zürich in 2019 [27] [11]. Following that, the current issues of BAZO, i.e. the unreliable network and the vulnerable consensus mechanism had to be tackled, along with further improvements to the scalability and security of the blockchain. During those steps, the implementation was frequently tested on a local testbed.

The nature of this thesis is highly experimental, and a variety of possible designs were brought up, discarded and/or refined together with the supervisors in the process. The structure of this work reflects that, by chronologically following the most important steps taken in order to reach a convincing result in the end.

1.4 Thesis Outline

This Bachelor thesis firstly contains an Introduction in Chapter 1. Afterwards, in Chapter 2, relevant related work is layed out. Chapters 3 and 4 then present how the projects of *Transaction Aggregation* and *Sharding* were merged and how the network and scalability issues were tackled. Chapter 5 and 6 afterwards propose a new consensus mechanism along with its implementation in order to overcome some of the most critical issues of BAZO. This implementation is tested and evaluated in Chapter 7. To round off the thesis, a summary is provided in Chapter 8.

Chapter 2

Related Work

This Chapter contains information about recent work which is relevant for this thesis, namely the implementation of both *Transaction Aggregation* [27] and *Sharding* [11], and relevant projects from the industry. For brevity, only the most important aspects about these are presented in this section.

In an effort to improve the scalability of the BAZO blockchain, the two strategies of *Transaction Aggregation* and *Sharding* were merged into one project as the first milestone of this thesis, which will be described in more detail in Chapter 3. In a later stage, the related work will play an important role when it comes to making BAZO more scalable and trustworthy by designing a new consensus protocol.

2.1 Transaction Aggregation

Transaction Aggregation was added to the BAZO domain in 2019 as part of a Bachelor Thesis at the University of Zürich [27]. In the following Section, the motivations behind the new mechanism will be explained, followed by a short description of how it was designed and implemented.

2.1.1 Motivation

Transaction Aggregation in its essence means the aggregation of several separate transactions into one aggregated transaction such that they are perceived as one unified transaction by the blockchain [27]. Since BAZO only stores the hashes of the included transactions in its blocks [30], with *Transaction Aggregation*, the space that transactions need in a block would drastically decrease. This is the case because all aggregated transactions would be put into one unified transaction, and then only the hash of this unified transaction will be put into the block. Since more transactions can be put into one block, the amount of validated transactions per second is also expected to increase [27].

2.1.2 Design & Implementation

The aggregation mechanism is designed in such a way that transactions which have the same sender, or transactions which have the same receiver, can be validated and grouped into one unified transaction [27]. A schematic representation of how this is done can be found in Figure 2.1.

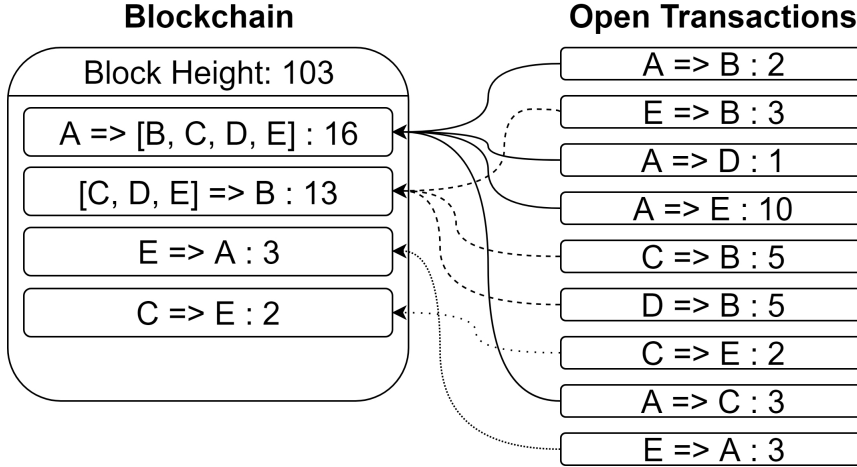


Figure 2.1: Concept of Transaction Aggregation [27]

As it can be seen, the pool of open transactions on the right hand side consists of 9 Funds Transactions of the form *Sender => Receiver: Amount*. With the help of the *Transaction Aggregation* mechanism, it is possible to store all these transactions using just 4 transactions in the block. This was achieved by designing an algorithm which enables validating as many transactions as possible while listing as little transactions as possible inside the block by inserting aggregated transactions [27].

2.2 Sharding

Just like *Transaction Aggregation*, *Sharding* was implemented on top of BAZO in 2019 at the University of Zürich [11]. In the following Section, the motivations behind this new mechanism will be explained, followed by a short description of how it was designed and implemented.

2.2.1 Motivation

Sharding in the domain of blockchain technology means to partition the transactions in a pool of transactions into smaller pools such that they can be processed in parallel [13]. The idea stems from the field of database systems where *Sharding* denotes the horizontal partitioning of a database among multiple physical data stores [16]. While the basic idea stays the same, there are various ways how a blockchain may implement *Sharding*, such as Zilliqa's protocol [10] or BAZO's *Sharding* protocol [11]. In theory, blockchain sharding

could lead to the throughput of the blockchain increasing linearly with the number of miners added to the blockchain, which makes *Sharding* one of the main topics whenever the scalability of a blockchain has to be improved, as it is the case with Ethereum [15].

2.2.2 Design & Implementation

First of all, in his Master Thesis, Kürsat Aydinli proposed a *Sharding* mechanism which would support putting multiple miners into one shard in an effort to minimize the likelihood of 51% attacks. Unfortunately, however, only an implementation which supports no more than one miner per shard was provided [11]. This is the implementation which was used in the further Chapters of this thesis.

The *Sharding* mechanism was designed in a way such that the global transaction mempool is partitioned according to the sender of the individual transactions. Each *shard* then mines a block for each height and adds it to the blockchain [11]. A representation of how the blockchain then looks like can be found in Figure 2.2.

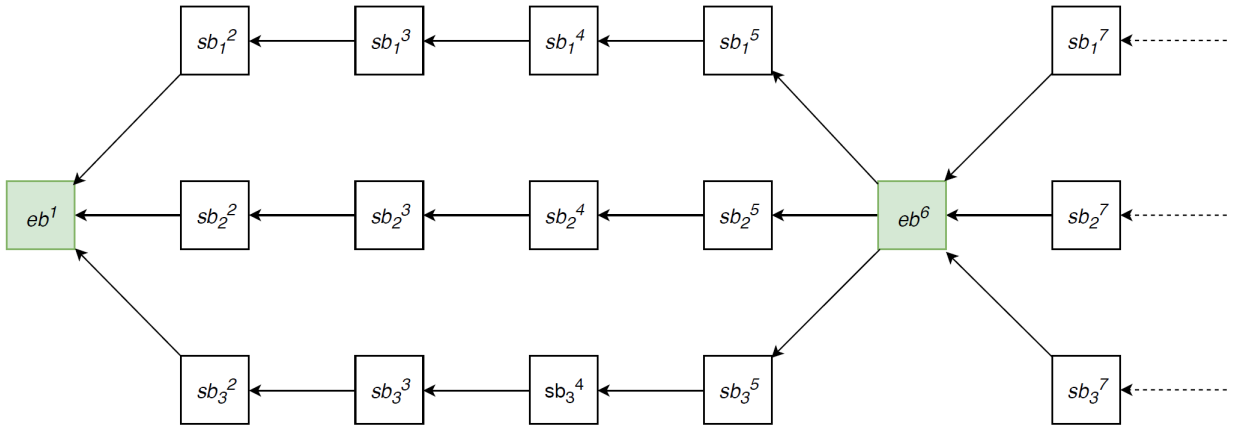


Figure 2.2: Concept of Sharding [11]

In order to make sure that all miners are updated about all validated transactions, even about those which don't belong to their shard, a new consensus protocol was brought forward. The main idea is that at a certain epoch height EH , a so called epoch block is inserted, in Figure 2.2 denoted as eb^{EH} , which consolidates the global state of the blockchain and re-assigns the *validators* to the shards in a randomized fashion. Then, Shard_i with shard ID i produces the block sb_i^h at height h . Using a proprietary synchronization mechanism, at each height h , all shards synchronize the global state with each other in order to have the global state before mining the next block with height $h+1$ [11].

2.3 Further Literature

The following Section consists of an overview of recent projects which are relevant to this thesis and BAZO in general. Instead of providing a full summary of the respective

projects, the emphasis lies on the points which heavily relate to BAZO and its future development.

2.3.1 Zilliqa

Zilliqa is one of the most popular working blockchains which implement the *sharding* feature. It has been mostly developed by computer scientists from academia [19].

In practice, Zilliqa's performance increases whenever 600 additional nodes join the network. This is caused by the sharding mechanism, which assigns roughly this number of nodes in each shard. The performance benefit is almost linear in the number of shards, until the network throughput bottlenecks the system when around 1 million nodes are active in the system. A testbed with 1800 active nodes manages to reach a TPS of 1218 as of October 2017 [19].

Zilliqa implements a hybrid consensus mechanism. For authentication of the nodes, it implements a Proof-of-Work mechanism in an attempt to combat sybil attacks [19]. For consensus inside the shards, however, Zilliqa employs a protocol based on Byzantine Fault Tolerance (BFT). The reason for this is because BFT protocols, once executed, provide instant finality. This means, that the blocks which are mined within the shards are instantly final and can't be changed anymore. As a consequence, mechanisms requiring to find the longest chain, as well as mechanisms to rollback are unnecessary [20]. Due to its finality attributes, many blockchain related projects in recent times decided to adapt BFT, such as Tendermint [8], NEO [29] and Libra [4].

The whole blockchain is managed by the DS Committee. At the beginning of each epoch, a set of nodes is elected to participate in the DS Committee, which assigns incoming transactions to the shards. Just like in BAZO, transactions are sharded based on the address of the sender [11] [19].

2.3.2 RapidChain

RapidChain is a blockchain proposed by researchers at Yale University, Dfinity and Visa. It proposes a scalable blockchain protocol, involving a sharding mechanism, and it is based on the UTXO paradigm [33].

The main idea behind RapidChain is to create multiple Committees, which are similar to shards in other projects. During bootstrap, a reference Committee is elected which is in charge of partitioning the set of nodes in the blockchain into separate Committees. Note that each node only stores $1/k$ of the blockchain, where k denotes the total amount of Committees in the system. It is claimed that using the methods proposed by RapidChain and running 4000 nodes, a TPS of 7380 could be reached [33].

The Committees have to be reconfigured after each epoch. In order to perform the reconfiguration of the Committees as efficient as possible, and to easily add new nodes to the blockchain, an adapted version of Harmony's Cuckoo Rule was implemented [33] [24]. In

short, the Cuckoo Rule, in the domain of state sharding, makes sure that the least amount of nodes have to be reshuffled. This is desirable, because with state sharding, nodes don't have the full view of the blockchain's state anymore. In the case of a reshuffle, therefore, the state of other nodes has to be downloaded in order for a validator to be accommodated into a new shard [24].

2.3.3 QuarkChain

QuarkChain proposes a two-layered approach. One layer consists of the elastic sharded blockchain. Elastic in this context means that the amount of shards and nodes in the blockchain can vary. The second layer consists of the root blockchain which is in charge of confirming blocks from the shards [14]. Obviously, the big challenge is to have a computationally strong enough root layer in order not to bottleneck the first layer.

Recently, the mainnet of QuarkChain was ported from Python to Go. In an interview conducted in November 2019, the CEO of QuarkChain, Dr. Qi Zhou, stated that the current performance of the Go build was 5-10 times faster than it was using Python. For reference, the highest TPS that could be reached using Python was 55'039.58 TPS which already put QuarkChain among the most performant blockchains around. The final goal, however, remains to reach 1 million TPS [31].

2.3.4 OmniLedger

Just like BAZO [30] and recently QuarkChain [31], OmniLedger was implemented in Go [23]. It was mainly developed by scientists at EPFL and aims to develop a secure, scalable and fully decentralized blockchain using sharding. Just like many other recent projects in the domain of blockchain sharding, it uses the UTXO model to process transactions [23] [33] [26]. In order to combat sybil attacks, it operates an identity blockchain which runs in parallel to the actual blockchain where the transactions are processed [23].

Similar to BAZO, it utilizes State Blocks as stable checkpoints which summarize the entire state of the blockchain and for the assignment of validators to shards [23] [11]. The shards in OmniLedger are called *Optimistic Validators* because they quickly validate transactions and put them into a block, creating a commitment, which with a high probabilistic likelihood won't change. Those blocks will then be checked again by the *Core Validators* in a scheme denoted as "trust but verify". Once accepted by the *Core Validators*, the blocks produced by the *Optimistic Validators* are final. The *Core Validators* process blocks in parallel in order to maximize the scalability of the system [23].

In favorable conditions, that is if it can be assumed that the amount of adversaries is low and with 1800 miners, OmniLedger reached 13'000 TPS. In a more realistic scenario, with more expected adversaries and also with 1800 miners, OmniLedger was still able to reach 4000 TPS [23].

2.3.5 Libra

Libra, Facebook’s ambitious cryptocurrency project, aims to completely change how financial services work in the future [28]. Unlike BAZO, it is based on the UTXO model [30] [26]. At launch, it will be a permissioned blockchain [26], which means that there is an additional layer of security. This is achieved by only letting a subset of the nodes perform certain critical actions on the blockchain [18]. It is, however, intended to move to a Proof-of-Stake consensus model in order to make Libra permissionless [26].

Consensus is reached by using a modified version of BFT, called LibraBFT, which should avoid forking [4] and therefore provide finality to validated transactions once the BFT mechanism is executed, which is an important feature for the problems that Libra is trying to solve. However, Libra does not involve sharding mechanisms and every node connects to all other nodes as well as keeping the global state [26]. In December 2019, a research team from Peking University reached around 1348 TPS with the Consensus Layer turned off, and about 744 TPS with the Consensus Layer turned on. The difference between the two scenarios is that with the Consensus Layer turned off, only one node was in the network meaning that it can execute the consensus mechanism itself. Consensus Layer turned on in this context means that there are 4 validators in the testbed. Overall, since there are no scalability mechanisms such as sharding in place, the performance decreases with every node that is added to the network [34].

2.3.6 Literature Summary

As it can be seen from the literature, there is a strong push for more scalable blockchains by implementing the technique of sharding. However, in order to keep the security of the system at the same level as if it was not sharded, additional measures had to be taken. Libra, although not sharded, takes perhaps the most drastic approach by developing a (at least in the start) permissioned blockchain. This means that trusted nodes in Libra will have special permissions, i.e. validating transactions and adding blocks [17].

A similar approach is taken by QuarkChain with its two-layered strategy. Here, the root layer checks the work of the shards [14]. OmniLedger works almost analogously, with the *Core Validator* pool concurrently validating the work of the *Optimistic Validators* [23]. Also RapidChain has such an implementation, where a small part of the original validator pool gets selected to join the reference Committee, which is in charge of assigning nodes to the shards [33]. Finally, Zilliqa employs a DS Committee which is in charge of assigning incoming transactions to the shards [19].

For the most part, the idea behind having a subset of validators with special responsibilities is to have a higher resiliency against potential attacks, and to increase scalability. The strategy which is implemented by Libra [17] is to make the blockchain permissioned, which increases trust in all validators, making much validation and verification unnecessary. The other strategy, which can be seen with OmniLedger, is to give power only to a small subset of the validators in the network. This forces the potential attacker to control a large part of the total network in order to have a realistic probabilistic chance to successfully act maliciously in the network [23].

Similar scalability and safety concerns will come up in BAZO as this thesis unfolds, and the insights into the previously described projects proved helpful to overcome those problems.

Chapter 3

Combining Transaction Aggregation & Sharding

Both the *Transaction Aggregation* [27] and the *Sharding* [11] mechanisms proved to be successful in increasing the scalability of BAZO. For this reason, as a first step for this work, it was decided to merge the two projects and find ways on how to accomodate those two mechanisms in one combined project. Fortunately, the concepts of *Sharding* and *Transaction Aggregation* don't contradict each other directly, so only few compromises had to be taken while merging the two projects.

Since Fabio Maddaloni's project contains more bugfixes and fewer errors [27], the approach was to implement the *Sharding* mechanism starting from his codebase. In this Section, firstly a step back will be taken and the consequences of having *Transaction Aggregation* and *Sharding* working together will be shown. After that, issues that arised during the merging and how they were solved are presented.

3.1 Overview

Even though the merged version of BAZO, which is how the build will be referred to from now, is based on the more stable codebase of Fabio Maddaloni [27], the consequences from implementing the *Sharding* [11] mechanism are much more far-reaching than the consequences of implementing the *Transaction Aggregation* mechanism. This includes the all-new synchronisation protocols as well as the partitioning of transactions onto shards. In fact, both projects have different consensus mechanisms in place, and to a great part the mechanisms from the *Sharding* project had to be taken over. This stems from the nature of how *Sharding* works. The mechanism of how to find consensus has to be distributed to several nodes contributing at the same time, other than the classic approach where only one block is appended to the blockchain at the time.

The first thing that led to problems was the network sharding, i.e. the partitioning of transactions into the respective shards. It doesn't let the *Transaction Aggregation* mechanism find a global maximum in all cases for the best aggregation anymore, since the

partitioning isn't done based on how many transactions could potentially be aggregated in one shard, but based on the address of the sender alone. This means that even if all open transactions have the same receiver, they would still be distributed among all shards. Based on this knowledge, it was decided to only aggregate transactions according to the sender. This effectively creates a one-to-many relationship between senders and receivers inside aggregated transactions and greatly simplifies the process.

Moreover, there is no working rollback mechanism anymore and if a node crashes, the blockchain has to be started from scratch. This is owed to the consensus mechanism implemented in the *Sharding* project. In his thesis, Kursat Aydinli presented possible strategies on how to overcome this problem [11]. However, in the scope of this thesis, only the status quo is considered. This means, that there won't be any more rollback mechanisms in place for the future development in BAZO, as it is the case for many other recent blockchain projects as in Section 2.3.

3.2 Solved Issues & Bugs

In this Section, the issues which were solved in the merged version of BAZO will be presented. They are just an excerpt of the most interesting issues which appeared either due to bugs from one of the previous projects or they were newly introduced due to code compatibility issues.

3.2.1 Decreasing the Complexity

In order to keep the complexity at an acceptable level, some simplifications in the *Transaction Aggregation* project had to be taken. Firstly, the doubly linked blockchain feature was removed because rollbacks don't work at the moment and its benefits were only small in the first place [27]. Moreover, as it has already been mentioned, transactions are only aggregated according to the sender's address, which is a limitation that comes naturally from the partitioning of transactions in the *Sharding* protocol, where transactions are already distributed to the respective *validators* according to the sender's address, and not the receiver's address. Finally, transaction count checks were removed on the validator side. While those checks are certainly useful to keep integrity and to combat replay attacks, they substantially slow down the validation process, especially with respect to IoT data streams which may bear massive amounts of data. Since all those functionalities are removed from the code with comments, they will be readily available quickly if needed again.

3.2.2 Different Protocol

Since BAZO was at a very early stage when both *Transaction Aggregation* and *Sharding* were implemented, the two projects had some differences when it came to protocol or bootstrapping new nodes. For example, the public addresses in the *Sharding* project were

of the form [64]byte, while in *Transaction Aggregation* they were of the form [32]byte. Furthermore, the bootstrapping processes were different, with the *Sharding* project relying on a genesis epoch block to bootstrap the blockchain, which the *Transaction Aggregation* didn't need to. In the end, a solution had to be found which accomodated both projects best.

3.2.3 General Sharding Bugfixes

The *Sharding* protocol itself revealed a fair amount of bugs. As an exemplary issue, in this Section, the structural flaws in the state transition mechanism will be explained, as well as describing how it was fixed.

A problem that the *Sharding* mechanism had was that state transitions didn't include the information if a node changed to a staking node, and the state transition didn't include the public commitment key of the newly staking node. For nodes which don't validate the blocks in which the staking transaction is processed, this means that they don't get the information about which node can stake as well as the key to validate their blocks. In the use case as of now, especially the information about which node is staking is essential for all nodes in order to include all staking nodes in the shard assignment process. Also, the public commitment keys are going to be important in a later stage when blocks will get validated by other nodes.

3.2.4 Edge Cases in Sharding

The problems that will be listed in the following paragraphs refer to edge cases around the epoch block in the *Sharding* project. Edge cases denote the cases where the blockchain is in the process of transitioning from one epoch to the next. In a nutshell, the problem is that the system isn't prepared that some nodes are in two different epochs. The process of finding those bugs was long and iterative as this Section will show. The naming conventions for the following demonstration were taken over from the *Sharded* version of BAZO, and can be retrieved from Kürsat Aydinli's Master Thesis [11].

The problem is elaborated here within the 3 shards - 3 miners configuration. Consider the case where all miners are mining the last block before the epoch block. At this moment, Miner₁ is the *leader* in Shard₁, Miner₂ is the *leader* in Shard₂ and Miner₃ is the *leader* in Shard₃. After mining the last block, assume that Miner₁ and Miner₂ receive the state transitions from the other miners and continue with mining the epoch block, while Miner₃ is still waiting for the state transition from Miner₂ due to an unstable connection. Miner₁ fulfills the PoS condition and can insert the epoch block which is received by all miners: Miner₁ is now in Shard₃, Miner₂ is now in Shard₁ and Miner₃ is now in Shard₂.

This new validator assignment confuses Miner₃ whose shard ID is now 2. It already processed the transition from Shard₁, so it won't request it again. Its shard ID is now 2, so it won't request any transitions from Shard₂ anymore. Now there only remains shard ID 3 which is now associated with Miner₁. If Miner₁ gets a request for the state transition

of Shard_3 , it will answer the request with its own state transition which it produced before the epoch block, which was produced for Shard_1 . Miner_3 however already has this transition in its stash and will keep requesting, but it will never receive the wanted transition. At this point, the system isn't recoverable anymore and gets stuck because Miner_3 cannot request the transition from the right miner. Note that this specific use case is only one of multiple scenarios where something can go wrong when inserting an epoch block.

The problem at hand can be illustrated with Algorithm 1 which sends out requests for missing transactions and with Algorithm 2 which is responsible for answering transition requests from other miners. It's also important to keep in mind that all nodes keep listening for new epoch blocks at all times in parallel.

Now, for Miner_3 , consider in Algorithm 1 from the point where the previous description left off. ID 2 will be rejected after the for loop because it's the current shard ID. ID 1 will be rejected too because it has been processed already. Only ID 3 remains, and requests sent out to ID 3 won't give the desired transition because the *validator* who now has ID 3 used to have ID 1 which Miner_3 has already processed. Algorithm 2 illustrates that the response will indeed produce the wrong result.

Note that this scenario isn't the only one where the previous mechanisms failed. For example, adding a new *validator* after the epoch block will cause even more problems.

Algorithm 1: Requesting Missing State Transitions:

EpochMining([32]byte *hashPrevBlock*, int *heightPrevBlock*)

Input : [32]byte *hashPrevBlock*, int *heightPrevBlock*

Output: none

```

[1] ...
[2] forall id in shardIDs do
[3]     if id != storage.ThisShardID and id not already processed then
[4]         p2p.StateTransitionReqShard(id, lastBlock.height)
[5]         if stateTransition st found then
[6]             ...
[7]             apply st on local state
[8]             set st processed
[9]             ...
[10]        end
[11]    ...
[12] end
[13] end

```

It becomes evident that miners need a way to remember the shard IDs that they were assigned to for each block height and requests should always be made using the correct shard ID for each height, in order to avoid any confusion. Therefore, on the requester's side, there is a new variable called *ThisShardIDDelayed* which will only be updated after a new block has been mined, effectively forcing the request to have the correct shard ID

Algorithm 2: Response to a State Transition Request:

stateTransitionRes(*peer p , []byte $payload$)

Input : *peer p , []byte $payload$

Output: none

```

[1] ...
[2] read int  $shardID$ , int  $height$  from  $payload$ 
[3] ...
[4] if  $shardID == storage.ThisShardID$  then
[5]   | StateTransition  $st := storage.ReadStateTransition(height)$ 
[6]   | ...
[7]   | []byte  $packet := BuildPacket(st)$ 
[8]   | ...
[9]   | sendData( $p$ ,  $packet$ )
[10] end
[11] ...

```

at all times. This means, that the check in Algorithm 1 on Line 3 now checks against the delayed ID. In the use case that was previously discussed, the delayed ID is 3, which means that requests to the correct shard ID 2 are possible.

On the request's receiver's side, a map has been installed which remembers the miner's shard ID for each epoch. The check in Algorithm 2 on Line 4 was replaced with a check against the new map. That means, that whenever a request for a certain shard ID id at a certain height h is processed, it is checked whether the miner was responsible for id at height h . If yes, the appropriate state transition will be sent as an answer. If no, the request won't be answered. In the previously discussed use case, this means that Miner₂, even though it's now in Shard₁, can answer the request for the state transition of Shard₂ of the previous height.

3.2.5 Epoch Block Finality

As a premise for this whole project, as well as the *Sharding* project, epoch blocks have to be final and should not be changed after they have been created [11]. In this Subsection, the epoch block sortition process in the *Sharding* project will be explained first. After that, it will be shown how this system is flawed with regards to distributed systems with unreliable and unstable connections. In the end, there is a discussion about how the problem was solved in the merged version of BAZO.

The epoch block sortition in the *Sharded* version of BAZO works in an unconventional way. Since rollbacks for epoch blocks are by definition not possible, it was decided to accept the last epoch block that arrived to be valid in all cases. In order to make sure that all *validators* eventually accept the same epoch block, a timer was instantiated. After epoch block creation, the miners have to wait for 5 seconds before continuing the mining process. This, along with the redistribution of incoming epoch blocks, should make sure

that every *validator* accepts the last epoch block in the end, and when they start mining, all have the same epoch block that they attach their first block of the epoch to. It is easy to see that this mechanism will eventually lead to inconsistencies, which is what will be shown in the following example.

In a 2 shards - 2 miners case, consider Miner₁ and Miner₂ are both mining the last block before the epoch block. Miner₁ is the *leader* in Shard₁, and Miner₂ is the *leader* in Shard₂. Both miners succeed in mining the block and Miner₁ gets the state transition from Miner₂. After that, the connection between the miners gets disrupted. Miner₁ can now start mining the epoch block, broadcast it into the network and continue mining the block after the epoch block. Miner₂ on the other hand is still waiting for the state transition from Miner₁. If now Miner₂ receives the transition from Miner₁, be it through a rebroadcast from other nodes in a more complicated case or just by a short re-instantiating of the connection between Miner₁ and Miner₂, Miner₂ is unaware of Miner₁ producing the epoch block and mines an epoch block itself. At this point, the blockchain cannot be rescued anymore because according to the premises in the *Sharding* project, epoch blocks always have to be final. When the connection goes up again, Miner₂ will broadcast the epoch block to Miner₁, and the new epoch block along with its validator shard assignment will be taken over by Miner₁ *after* it mined the first block after the epoch block already. This means that one miner can have two different shard IDs within one epoch which makes the whole transition and request structures fail.

A large amount of time was spent analyzing the cause and finding solutions for this problem as it only starts to become really apparent with 5 or more miners in the network. Still, this was an issue which was solved by a pragmatic approach: The shard with shard ID 1 can produce the epoch block. Thereby, there is no race condition in the creation of epoch blocks anymore and there is immediate finality. Furthermore, this condition doesn't provide any more vulnerabilities than the previous system because the sortition mechanism for who can create the epoch block is still randomized in a similar fashion. The only disadvantage is that the PoS mechanism loses its weighing mechanisms. That is, nodes with a higher stake aren't more likely to mine the epoch block anymore. This problem, however, could be solved by redesigning the validator shard assignment process in the future. In the further topics of the thesis, however, this issue won't be discussed anymore.

The advantages of immediate finality are diverse, but mostly they enable the blockchain to be more scalable, because rollbacks are time- and resource-consuming. Moreover, it enables to process transactions in (quasi) real-time, whereas it is said that Bitcoin transactions take up to 60 minutes until they can be considered as final [21].

Chapter 4

Scalability Improvements

One of the main goals of this thesis is to improve the scalability of the BAZO blockchain. In this Chapter, the process of making BAZO more stable and performant will be documented.

4.1 Assessing the State of the Merged Version

Before taking further steps to enhance the scalability of BAZO, the merged version of BAZO was supposed to be thoroughly tested both on an online testbed as well as locally. Unfortunately, however, initial deployments even on a local testbed proved to be without much success. The reason for that was the problem with disconnecting nodes, as it has been discussed in the previous Chapter. Before being able to compile any meaningful data, it was therefore necessary to get the system to a level where BAZO would be testable. The first attempt involved adding more miners to a shard in order to minimize inter-shard communications. After that, the p2p package was analyzed more closely and further methods were tested and evaluated.

4.1.1 Reaching a Full Mesh Network

As a side note, the first big problem that was discovered while running the BAZO system locally according to the tutorial in the *README.md* file in the miner application was that each node only establishes a connection to the bootstrap node, and no other node, essentially creating a network with a star topology. In this configuration, communication was only possible by the rebroadcast mechanisms in the root node. Of course, this was not a satisfactory state for BAZO, as a full mesh was the network structure which was to be achieved. In a full mesh, every node would be able to communicate with every other node in the system, eliminating the need to rely on any third nodes in case of direct communication. This flaw is graphically represented in Figure 4.1.

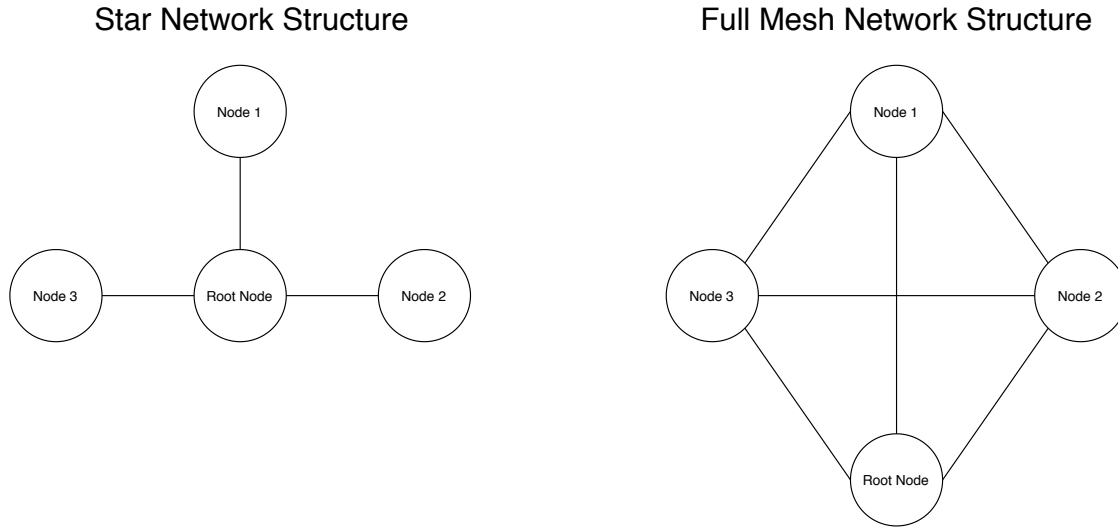


Figure 4.1: Star and Mesh Network Topology

After analyzing the p2p package, the problem became clear. Whenever a new node is started, it is given the address of the root account, which it can connect to immediately. After that, the node periodically starts sending out *Neighbour Request* messages, requesting a list of addresses of all the nodes which are currently online in the system in order to connect to them. The problem, however, is that the *Neighbour Request Response* mechanism only supports IPv4 addresses.

And this is exactly where the problem lied. If the nodes in BAZO are started using *localhost* as the IP address, as it was recommended in every documentation of how to run BAZO, especially in the *README.md* file, these *Neighbour Request Response* messages will always be empty, because there are no proper IPv4 addresses at hand.

The problem was solved by taking a pragmatic approach and adjusting the *README.md* file, which serves as the only documentation of how to run the system. Nodes are now started using the IPv4 version of localhost, which conventionally is *127.0.0.1*. In the future, however, the *Neighbour Request Response* mechanism could be extended to not only support IPv4 addresses, but also IPv6 and the commonly used keyword *localhost*.

4.1.2 More Miners per Shard

Once the full mesh network structure was achieved, in theory all nodes could communicate with all other nodes in the system. Due to the network problems, however, many of these connections between the nodes were susceptible to disconnects. The first attempt to make the system more reliable was to add more miners to one shard. The idea behind adding multiple miners into a shard was to improve scalability by having smaller but more synchronization steps. In theory, having multiple miners in one shard allows BAZO to have unreliable connections between shards, only requiring the nodes within a shard to synchronize first and then sending just one aggregated transition to the network in order to synchronize the shards as a whole. With the two-step synchronization process proposed

in this Section, the network is effectively partitioned, minimizing the need for *inter-shard communication* and messages that have to be reached by all network members.

Design

The design of the *multiple miners per shard* version is very similar to the one of the merged version, with the addition of a new type of transition and a new synchronization mechanism. Those design choices will be briefly presented in the following Subsections, for brevity not going into all technical details, but empathizing on the most interesting aspects.

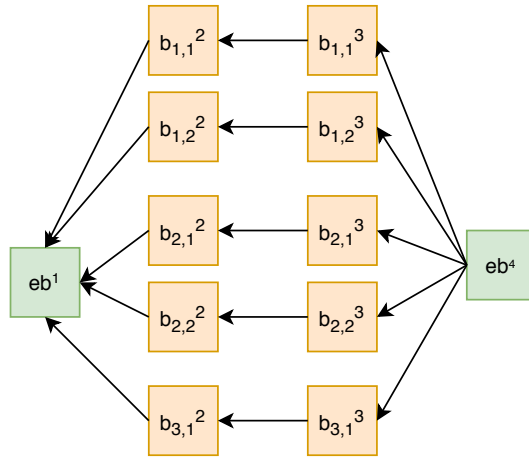


Figure 4.2: Schematic Representation of the *Multiple Miners Per Shard* Version

Figure 4.2 shows a schematic representation of how the *multiple miners per shard* version works on a high level. The quadratic boxes all represent different kinds of blocks within the blockchain. Those labeled eb^i are *epoch blocks* and those labeled $b_{j,k}^i$ are *blocks*. The variable i denotes the block height, whereas the variables j and k identify shard ID and block ID, respectively.

For demonstration purposes, a blockchain with epoch length $length_{ep} = 2$ was chosen. The blockchain starts with *epoch block* eb^1 . Startig from there, all *validators* chain the blocks that they mine to the initial *epoch block*, and so on, as denoted by the arrows.

Protocol

In order to reuse as much code as possible from the merged version of BAZO, as many of the current mechanisms as possible have been kept in the merged version. The most notable changes which had to be made are:

1. New entity management
2. New mechanism to assign miners to shard IDs and block IDs

3. Addition of the block transition
4. New transition management

The following sections aim to explain how those additions were implemented in BAZO and what consequences they bring to the system.

Entity Management

Entities in the *multiple miners per shard* version are handled in a similar fashion compared to the *sharded* version of BAZO [11]. There are still the three old types of entities in the protocol, namely *users*, *validators* and *leaders*. *Users* are non-staking nodes, so they don't participate in the mining and validation process. Still, because of the *network sharding* mechanism that was implemented, *users* or their transactions respectively will be virtually placed onto a specific shard for each epoch. In summary, the responsibility of *users* only includes issuing transactions into the network. *Validators* on the other hand now take a similar role as the one of *leaders* in the *sharded* version of BAZO. This means, that they are responsible for the definite mining of blocks as well as the distribution of their *block transitions*. Using the distributed *intra-shard communication* protocol, they are all together responsible for the integrity of their shard. This means that they collect all *block transitions* from the same shard and block height and aggregate them to a *state transition*. This *state transition* will then be sent out by all *validators* of the shard, maximizing the chances to reach all the other shards even if the network is unreliable. Note that since all *validators* in one shard process the same *block transitions*, their produced *state transitions* will be the same. The *leader* is responsible for the integrity of the whole blockchain and is in charge of mining the *epoch block* at the end of each epoch. All staking nodes are *validators*, and the *leader* formation will be explained in the next Section.

In the current build of the *multiple miners per shard* version, *users* always stay in their shard, based on the hash of their public address. *Validators*, on the other hand, get shifted around the different shards based on the assignment in the *epoch block*. The *leader*, who is a subset of the *validators*, is exactly the *validator* who got assigned to mine the *epoch block*.

Epoch Block Finality

As it was mentioned before, epoch block finality is crucial and it is reached by letting the miner in shard 1 create the epoch block in all cases in the merged version of BAZO. In the *multiple miners per shard* version, however, shard 1 may contain multiple miners, so it was decided to extend the epoch block creation condition to only let miner with block ID 1 in shard 1 mine the epoch block, thus making it the *leader*.

This leads to a well-defined distribution of responsibilities and there is no danger of rollbacks anymore, which aren't considered at this stage of the thesis.

Assignment Mechanism

In order to still be able to partition the total transaction pool among all *validators* in the system, *validators* inside a shard have to be numbered. If all *validators* have both a shard ID and a block ID, they can be uniquely identified as entities in the system. As a consequence, the total transaction pool is not only partitioned according to the Shard ID anymore, but also according to the block ID of the *validator*, leaving each *validator* with its own partition of the total transaction pool. The assignment mechanism takes place during the creation of an epoch block with Algorithm 3, which is a modified version of the assignment algorithm used in the sharded version of BAZO [11].

Algorithm 3: Assignment of Miners to Shards and Blocks to mine:

AssignValidatorsToShards()

Input : none

Output: map[[64]byte]int *validatorShardAssignment*, map[[64]byte]int
nodeToBlockAssignment

```

[1] ...
[2] create validatorSlices
[3] create validatorAssignedMap
[4] forall account in storage.State do
[5]     if account.IsStaking then
[6]         validatorAssignedMap[account.Address] = false
[7]         validatorSlices = append(validatorSlices, account.Address)
[8]     end
[9] end
[10] ...
[11] for j := 1; j <= validatorsPerShard; j++ do
[12]     for i := 1; i <= numberOfShards; i++ do
[13]         if len(validatorSlices) == 0 then
[14]             return validatorShardAssignment, nodeToBlockAssignment
[15]         end
[16]         randomIdx := rand.Intn(len(validatorSlices))
[17]         randomValidator := validatorSlices[randomIdx]
[18]         validatorShardAssignment[randomValidator] = i
[19]         nodeToBlockAssignment[randomValidator] = j
[20]         validatorSlices = removeValidator(validatorSlices, randomIdx)
[21]     end
[22] end

```

As one can see, the algorithm first creates the *validatorSlices* to store all staking nodes and the *validatorAssignedMap* to keep track of which nodes have already been assigned. The two for-loops then assign each node one-by-one to a shard ID and a block ID. Note that although this mechanism is iterative, the node chosen at each step is randomized on Line 16 which minimizes potential attack threats.

Block Transition

The *block transition* is the element which enables *intra-shard communication*. *Block transitions* are produced at each height after a block has been mined by a *validator* and they are sent out to the shard in order to synchronize the other *validators* in it. Similar to the *state transition* taken over from the merged version of BAZO, which is responsible for the *inter-shard communication*, it mainly contains the *relative state* along with the hashes of all transactions which have been validated in the block that was previously mined. The main difference, other than implementing a completely new protocol, is that *block transitions* also carry a *block ID* to identify which *validator* within the shard produced it, and *block transitions* carry the *relative state* of a single block, whereas *state transitions* carry the *relative state* of the whole shard.

Transition Management

BAZO aims to be a global blockchain network, and the way how both *block transitions* and *state transitions* are managed within the network is crucial to the success of the system. While *state transitions* are meant to be sent out through the whole network and are designed for *validators* from other shards as well, *block transitions* are specifically designed to be created, distributed and processed within a specific shard, according to the *intra-shard communication* protocol. While BAZO's p2p network package doesn't implement a *multicast* function to send out messages to select *validators* inside a specific shard, *block transitions* can be checked at arrival if they reached the correct shard. By doing that, *validators* only have *block transitions* from their own shard in their stash, which is important for the state synchronization. This is implemented in Algorithm 4 as follows:

Algorithm 4: Processing of Block Transitions after reception:

`processBlockStateData([]byte payload)`

Input : []byte *payload*

Output: none

```
[1] decode BlockTransition bt from payload
[2] ...
[3] if bt not in ReceivedBlockTransitionStash and bt.ShardID ==
    storage.ThisShardID then
[4] |   store bt in ReceivedBlockTransitionStash
[5] |   broadcast bt to the network
[6] end
[7] ...
```

State Synchronization

The synchronization mechanism of the *multiple miners per shard* version can be split up into *intra-shard communication* and *inter-shard communication*. In fact, it's an extended version of one found in the merged version of BAZO.

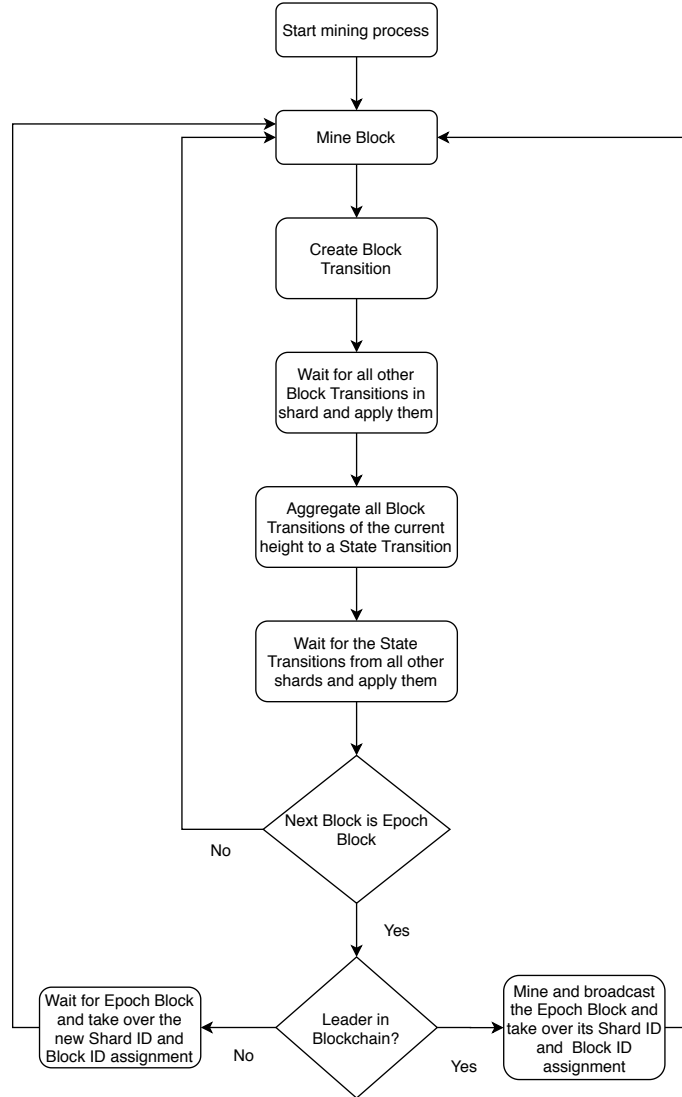


Figure 4.3: Flowchart of the *Multiple Miners Per Shard* State Synchronization

The mechanism is drafted in Figure 4.3 as a Flowchart: Note that since stake transactions are only processed right before a new epoch block, whenever a *validator* starts the mining process the first time, it will always be right after an epoch block. Therefore, the check whether the new node has to mine an epoch block is unnecessary. Firstly, the *validator* takes all the transactions which have been partitioned to it. Afterwards, an optimal set is put into the block and validated, according to the *Transaction Aggregation* mechanism. Then, the *intra-shard communication* takes place, in which the shard synchronizes to get its local state. There, it can apply all the block transitions that have arrived for the current height and shard. Once all *block transitions* from the shard have been processed, the *intra-shard communication* is finished.

Moreover, whenever a *validator* finished this process, it creates the *state transition* and sends it out to the network. Since this creates redundancy, and the *state transition* will be distributed as soon as it's produced by the first validator in the shard, it will hopefully accelerate the *inter-shard communication* and with that the whole synchronization process. After all those transitions are received, the *intra-shard communication* finished and the globally correct state of BAZO is established.

Since the blockchain is partitioned into epochs, whenever the next block height will be an epoch block, the *leader* node will create the epoch block with the globally valid state and the new shard ID and block ID assignments.

Evaluation of Multiple Miners Per Shard

As it became clear during the local testing of the *multiple miners per shard* version, adding more miners to a shard unfortunately did not bring the hoped for result of a more reliable network. Connections between the miners stayed unstable, and splitting up the communications into *intra-shard communication* and *inter-shard communication* did not allow for BAZO to run more smoothly. The reason for this is rather simple: As soon as a disconnect happens, the whole blockchain synchronization mechanism is halted until the connection is reinstantiated. Since disconnects still happen too often, and were not averted by this new implementation, a new approach was necessary in order to obtain a testable version of BAZO.

4.1.3 Network Improvements

As a next development step, it was clear that the p2p package had to be tackled directly in order to obtain satisfying results.

In his Bachelor Thesis, Fabio Maddaloni already discovered the issue of the unreliable p2p network. In an attempt to improve the stability, the following steps were taken [27]:

1. Analyzing the cause of the disconnecting nodes.
2. Fighting the symptoms of the disconnecting nodes.

Step 1 has already been discussed thoroughly in Fabio Maddaloni's thesis [27]. In short, it has been concluded that the problem lies in concurrency during reception of incoming messages. Step 2 then builds up on step 1 and attempts to increase the likelihood of successfully sending a message through the unreliable network by re-sending messages which could not reach their destination. This mechanism will be discussed to greater detail in the following Section.

Not Received Messages

At this point, the main stability improvement for the p2p package from Fabio Maddaloni's thesis comes into play: The rebroadcast of messages that could not be sent to other nodes while a connection was down. In short, for each node in the network a slice of unsent messages was kept in memory. As soon as a connection between two nodes could be re-established, all the messages that could not be sent through while the connection was down are re-sent at once [27]. This makes sense in the environment where the thesis was written, but it's not suitable for the merged version of BAZO. This is due to the inter-shard synchronization that happens at each block height. In almost all of the cases, only the most recent messages are required anyways, so sending a slice of old messages as well only decelerates the system and makes it more error-prone, because every sent out message can potentially lead to a wrongly received message and therefore directly cause a disconnect. If the lost message was a request, for example, another request will follow anyways which is guaranteed by the construction of the synchronization mechanism. Moreover, it is important to remember the main reason why nodes disconnect from each other in the first place, which is the sending of messages with a wrong Header Type ID [27]. Those messages will be completely lost, but they aren't remembered as unsent by the sender, leaving an incomplete picture of what has been sent and what has not.

Disabling the sending of unsent messages brought some improvement, but obviously it was not the cause of the disconnects in the first place. While analyzing the network and the causes of the disconnects, it could be observed that at certain times, a broadcast by a sender leads to a corrupted received message at all receivers of the message. Moreover, those messages all held *identical* and *wrong* information. This error can therefore not be explained by concurrency problems at the reception of messages anymore, but it has to be explained by errors at the sending of a message. In the following Section, therefore, this phenomenon will be analyzed more closely.

Corrupted Messages

Now, as it has been mentioned in Fabio Maddaloni's thesis [27], there is a check at the sender side, which makes sure that the message that will be sent out is not corrupted. After that, the message is converted to a byte stream and prepared to be sent out to the network. This means that, looking at the case which was presented before, the message is corrupted after its creation, but before it is received by the receiving node.

Inside BAZO, all of the messages which are broadcast to the network are handled by the same goroutine called `go minerBroadcastService()`. Inside this goroutine, a channel called `minerBrdcstMsg` which is filled up by the messages to be broadcast is continuously consumed which spawns another goroutine called `go sendAndSearchMessages(msg)`. It is possible that some of the concurrency problems were caused in this part of the code, since inserting a timer to the method which forces the system to wait 5 milliseconds between consuming from the `minerBrdcstMsg` channel made the system more stable.

The reason for this assumption is that whenever a message has been created which should be broadcast to the system, it is immediately written to `minerBrdcstMsg`. The goroutine

`go minerBroadcastService()` is therefore the last step involving concurrency before the message is finally sent out to the network.

Additionally, it was observed that the network had particular issues with working properly when the network congestion peaked, i.e. when lots of transactions were sent through it. In an attempt to decrease the congestion, it was therefore decided to disable the rebroadcasting of transactions, also with the background of the delay of 5 milliseconds being too much if thousands of transactions are sent to the network per second, making throughput too low.

Conclusions

On one hand, the measures taken to improve the network reliability were successful in a way that disconnects barely happen anymore. The network was therefore stabilized. On the other hand, this meant that transaction rebroadcasts were impractical and impossible. Testing the merged version of BAZO on a truly decentralized system, other than testing it locally, therefore wasn't an option anymore for the time being. Still, the improvements taken finally afforded to test the theoretical scalability that the merged version of BAZO could offer in a local testbed described in the next Section, even though the exact root causes of the problems couldn't be identified with complete certainty and therefore weren't removed entirely. Effectively, mostly only the symptoms were cured, as opposed to the actual cause.

4.1.4 Testbed Implementation

In the scope of testing the scalability of the merged version of BAZO, it was decided to run the blockchain in a local testing environment, similar to the one which was used by Kürsat Aydınli in his Master Thesis [11]. This not only makes the blockchain more reliable and easier to test, but it also allows for a higher sending rate of transactions to the network [11], which in turn allows BAZO to reach higher TPS results. Moreover, since transaction rebroadcasts have been eliminated, a local testbed implementation affords for a reliable implementation of a mechanism where transactions can be sent to the ports of all *validators* without having to rebroadcast them upon reception by directly addressing each validator for each transaction.

The idea is to create artificial wallets in the miner application which can be used to issue transactions to the network. Since the transactions are created in a unit test located at `..\miner\sharding_test.go` inside the miner application, no separate port has to be opened to reach the nodes [11].

Firstly, the nodes used to validate are instantiated and connected to the running blockchain. Then, a fixed amount of new artificial wallets are created within the test program, filling each with enough funds to send the required amount of transactions through the local network. After this is done, the artificially created wallets will send a high amount of funds transactions through the network, all addressing the root wallet. This is achieved

by spawning a concurrent goroutine for each artificial wallet, and writing the generated transactions to each *validator*'s port separately. Note that since transactions are sharded according to the sender, sending all funds to the same receiver already creates an optimal testing environment for the nodes; sending the transactions to more different wallets would not make any difference.

Early implementations of this testbed saw a loss of almost 50% of all sent transactions inside the concurrent goroutines, with more transactions lost when more *validators* were connected to the network. This in turn hints at more p2p issues present in BAZO, this time at the receiving side of messages again, which won't be evaluated more closely at this state of the thesis. As a compromise, the current testbed now has one concurrent goroutine per *validator*. This means that for each *validator*, there is one goroutine supplying its port with transactions. This method sends out transactions faster than a fully iterative approach and all sent transactions reach their destination.

Since the *validators* all get supplied with transactions separately and there is no transaction loss from the testbed anymore, the rebroadcast of received transactions could be turned off and all *validators* still have all transactions in their local storage. This and the improvements mentioned in Section 4.1.3 reduced the traffic and improved the p2p package reliability in such a significant way so that the testbed is stable enough to be run with at least 4 *validators* running at the same time, validating and communicating concurrently.

From early analysis, it became apparent that especially the *Transaction Aggregation* mechanism is very powerful as it can aggregate a practically infinite number of transactions into one transaction which merely takes a size of 32 bytes in the blockchain, which is the same as any other transaction in the network. Leaving the blocksize at 5000 bytes would therefore allow for a theoretical amount of $\lfloor \frac{5000-658}{32} \rfloor = 138$ transactions in one block. It follows that with the current blocksize of 5000 bytes, more than 138 different artificial wallets need to send around transactions in the network in order for *Sharding* to make an improvement over regular *Transaction Aggregation* without *Sharding*. Note that transactions are only aggregated according to the sender in the merged version of BAZO. That means, that aggregated transitions are factually a one-to-many relation between the sender and the receiver of funds transaction. This issue will be touched on again during the test Section.

Furthermore, the goroutines were set up to send a batch of transactions from each virtual wallet first before moving on to the next batch. Doing that ensures that there are transactions from as many different validators in the network as possible, which prevents the *Transaction Aggregation* Mechanism from validating all transactions from one wallet at once, which would bias the testing results in favor for the non-sharded version of the merged version of BAZO.

The goroutine used to send around the transactions, after creating the virtual wallets and filling them up with sufficient funds, is shown in Algorithm 5. Before calling the concurrent goroutines, a timer is started to measure the sending rate of transactions and a waitgroup *wg* to wait for all goroutines to finish before the timer is stopped is instantiated. Remember that there is one goroutine for each *validator*. In the Algorithm, it can be seen that the variable *txPerRound* is the batch size. One batch is sent out for each virtual

wallet before moving to the next round, with variable *numberOfRounds* denoting the total amount of rounds. It is easy to see that the total amount of transactions sent per virtual wallet can be calculated in the following way:

$$\#txPerVirtualWallet = txPerRound * numberOfRounds$$

For all test runs, it was decided to create 30 artificial wallets within the unit test. The *txPerRound* was set to 15, and the *numberOfRounds* was set to 20. This means, that there will be a total of 90'000 transactions.

Algorithm 5: Concurrently sending transactions to a validator:

SendTransactions()

Input : none

Output: none

```

[1] int txPerRound := 150
[2] int numberOfRounds := 20
[3] //remove from waitgroup only if the goroutine finished with all transactions
[4] defer wg.Done()
[5] for i := 1; i <= numberOfRounds; i++ do
[6]     forall virtualWallet in virtualWallets do
[7]         for txSent := 1; txSent <= txPerRound; txSent++ do
[8]             build fundsTx from virtualWallet to root
[9]             send fundsTx to validator's IP Address
[10]        end
[11]    end
[12] end

```

4.1.5 Metrics Used

The metrics used in the evaluation process for the BAZO blockchain is a subset of the ones proposed in a previous thesis [27] and are defined as follows [32]:

- **TPS** = *Transactions per second* = Average number of transactions validated per second.
- **TPS_{sent}** = *Transactions per second sent* = Average number of transactions sent to the network per second.

While there are more metrics which could be used, those are the metrics which were defined to be the most interesting ones for the measurement of the performance and scalability of BAZO [27]. In later stages, more metrics could be re-introduced to get hints at where more improvements can be made.

TPS

The *transactions per second* are calculated with the help of the timespan between the first transaction sent to the network and the first block which includes all sent transactions, called the *timespanValidation*.

$$\text{TPS} = \frac{\text{numberOfValidatedTransactions}}{\text{timespanValidation}} = \frac{\text{numberOfValidatedTransactions}}{(\text{timeOfBlock} - \text{timeFirstTxSent})} \quad (4.1)$$

The *TPS* on its own does only tell how many transactions are validated in a blockchain per second. It does not reveal anything about limiting factors and why a specific *TPS* is reached.

TPS_{sent}

The *sent transactions per second* is the average number of transactions sent to the network by all clients and it indicates the upper limit for the number of transactions which can be validated per second.

$$\text{TPS}_{\text{sent}} = \frac{\text{numberOfSentTransactions}}{\text{timespanSent}} = \frac{\text{numberOfSentTransactions}}{(\text{timeLastTxSent} - \text{timeFirstTxSent})} \quad (4.2)$$

The *TPS_{sent}* indicates how fast transactions are sent to the network. When the *TPS_{sent}* and the *TPS* are close to each other, it can be assumed, that all transactions get validated shortly after they are issued. This because it does not take much longer to validate all transactions than it takes to send them. Thus, if they diverge a lot, it is an indicator, that it takes longer until the transactions are validated.

Furthermore, it is an indicator for the upper *TPS* limit, since the *TPS* can not be higher than the *TPS_{sent}*.

4.1.6 Testing Results & Evaluation

In order to assess how the *Sharding* Mechanism and the *Transaction Aggregation* Mechanism work together, the merged version of BAZO was tested in a 1 shard - 1 validator, a 2 shards - 2 validators and a 3 shards - 3 validators case. The results can be extracted from the following subsections.

1 Shard - 1 Validator

This configuration is theoretically equivalent to the *Transaction Aggregation* project with 1 *validator*. It's interesting because it shows how the sharding affects and hopefully improves the performance of the merged version of BAZO.

Table 4.1: Table of different Block Sizes influencing the *TPS* with 1 *Validator*

Block Size	Transactions			
	Bytes	#sent	#validated	TPS _{sent} TPS
800		90'000	90'000	918.4 301.0
1000		90'000	90'000	937.5 324.9
5000		90'000	90'000	810.8 334.6

As it can be seen from the testing results, the TPS stays relatively stable with respect to block size, while the TPS_{sent} is stable as well. At first glance, this might look surprising, keeping in mind that the transaction pool for each sending wallet is filled up continuously. A deep analysis of the log data however provides some explanation:

In the 5000 bytes run, the block carrying the most transactions had a total of 50'380 funds transactions in it, using 30 aggregated transactions in total. The total time needed from preparing the block until validation is finished is 158 seconds. Out of this time, 153 seconds are used for postvalidation, which mostly translates to deleting the transactions which were put into the block from the open transaction pool and writing them to the closed transaction pool. For this block, we therefore have a transaction management performance of 329 transactions per second, where transaction management means shifting transactions around the data stores. Note that this number is very similar to the actual TPS achieved in the test runs. The bottleneck of the system in its current state therefore is not how many aggregated transactions can be put into one block, but how fast the included transactions can be handled by the miner's database. Due to these findings and the discussions from the previous Section, it was decided to only use a block size of 800 bytes in the following test runs.

2 Shards - 2 Validators

In this configuration, there are 2 *validators* in the system, ready to validate incoming transactions. The results can be found in Table 4.2.

Table 4.2: *TPS* with 2 *Validators*

Block Size	Transactions			
	Bytes	#sent	#validated	TPS _{sent} TPS
800		90'000	90'000	1083.3 241.2

The result which can be observed from the 2 *validators* scenario might seem confusing at first. After all, the sharding mechanism is supposed to distribute the workload of the

blockchain to more entities and therefore enhance performance. Again, the inner workings of BAZO have to be considered in order to find an explanation for this behaviour.

In fact, the results from the first test run will help us understand the problem better; it was concluded that the bottleneck of the blockchain is how fast transactions can be handled by the miner. Now consider the sharded approach: At each step and for each shard, on a high level the following steps are performed:

1. The transactions to be put into the block are evaluated and aggregated
2. The transactions are validated and the block is mined
3. The transactions are deleted from open storage and written to closed storage
4. The state transition is created and sent to the network
5. The relative state from other shards is applied
6. The transactions which were validated in other shards are deleted from open storage and written to closed storage

The last point reveals to us that the problem lies in redundancy. In fact, each shard still has to handle each transaction in its database, leaving the whole sharding strategy with a negative improvement. The drop in TPS can therefore be explained with the inter-shard communication overhead which still prevails while the most time-consuming operations are performed on all shards separately. This becomes clear in Figure 4.4.

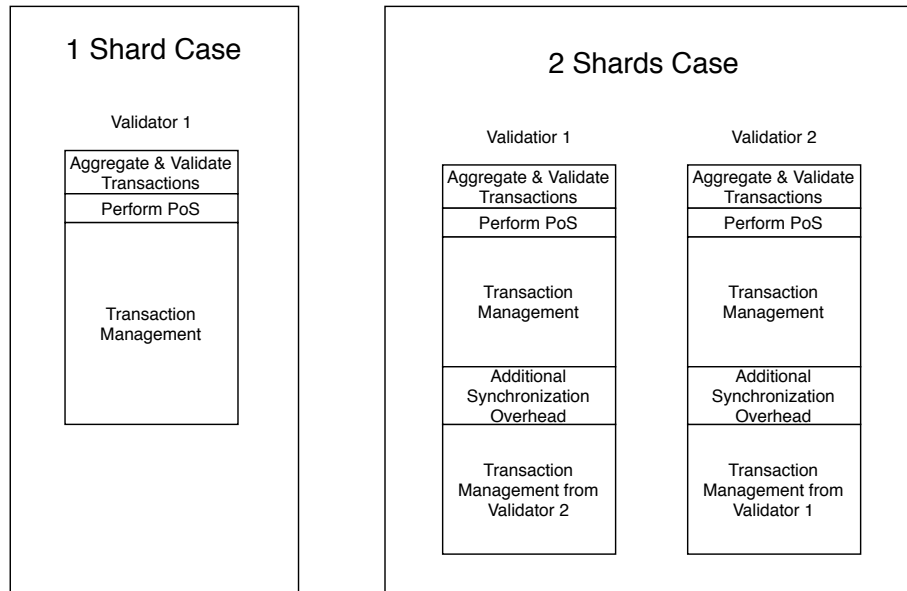


Figure 4.4: Schematic Representation of the Transaction Management Overhead in the *Validators*

Figure 4.4 shows the weakness that the merged version of BAZO has when it comes to decentralization and workload partitioning. For both a 1 shard case and a 2 shard case, it

roughly describes the time needed to perform a certain step that has to be executed within an arbitrary block height h . Note that for demonstration purposes, only an estimation of the time needed is used. Generally, larger the box, the longer the step inside the box will take. It's evident now, that if the *transaction management*, that is shifting transactions around data stores, takes by far the most time, which has been confirmed in the 1 shard - 1 validator test case, and if it isn't distributed among the shards wisely, it's not realistic to expect any significant improvements by adding more *validators* to the blockchain. This might also serve as an explanation for the surprisingly low scaling benefits when adding more miners in Kürsat Aydinli's Master Thesis [11].

Pursuing the same thought, the TPS drop is perhaps so drastic because the validation workload isn't perfectly distributed at all times, i.e. one shard might be less busy during one block height than another shard because it got assigned less transactions. However, it still has to wait for the other shards to finish before it can continue with the next block. This is an issue inherent to the *Sharding* mechanism in BAZO and how the transactions are partitioned. On top of this inevitable asynchronicity comes of course the communication overhead and the time it takes to send transitions around the network.

In an attempt to address these issues, the two-shard scenario was re-run with a twist. Instead of immediately performing the *transaction management* for transactions that were validated in other shards, it is deferred to the end of each epoch. Now if all transactions are validated within the same epoch, this will translate to a lower *transaction management* overhead and an increased TPS over the 1 shard scenario, because the *transaction management* of transactions validated by the other shard won't be considered by the TPS anymore, as it can be seen from Table 4.3. It is important, however, to keep in mind that this mechanism merely delays the inevitable point where the *transaction management* for the transactions from the other shard has to be performed. Still, it gives an indication of how the system could perform if the *transaction management* of transactions that have been validated in other shards could somehow be omitted. A design which solves exactly this issue will be presented in Chapter 5.

Table 4.3: *TPS with 2 Validators. Accelerated.*

Block Size	Transactions			
	#sent	#validated	TPS _{sent}	TPS
800	90'000	90'000	967.7	414.7

3 Shards - 3 Miners

The results from the previously proposed accelerated version of BAZO can be found in Table 4.4. The performance gain over the results found in the same test scenario with 2 shards - 2 miners is substantial, which shows that database operations and the transaction management are things to look into in order to further improve the scalability of BAZO.

Table 4.4: *TPS with 3 Validators. Accelerated.*

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	90'000	90'000	1071.4	523.3

Furthermore, as it can be seen in Table 4.5, the TPS for the normal merged version of BAZO has further dropped.

Table 4.5: *TPS with 3 Validators*

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	90'000	90'000	882.4	196.0

4.1.7 Testing Conclusions

The testing results clearly revealed the areas where the merged implementation of BAZO has its weaknesses. One of them, which is communication overhead, was already observed by Kürsat Aydinli in his Master Thesis [11]. Another one, which is *transaction management* overhead, meaning that transactions which don't belong to the own shard have to be processed in some way, was not observed there. This, however, is not surprising, because the amount of transactions being processed in one block were relatively low at around 500, which means that the *transaction management* should be finished in about a second. The third weakness, which closely relates to the previous problem, is database I/O speed was not discovered yet either. The fourth and last weakness is load balancing, which can be a problem when not all shards are tasked to validate the same amount of transactions. Then, other shards have to wait for the one shard with the highest workload to finish.

4.2 Attempts to Increase the Scalability

In the following Sections, the most significant efforts made to increase the scalability of the BAZO blockchain will be presented.

4.2.1 Database Improvements

As it was observed in Section 4.1.6, one of the main bottlenecks in the merged version of BAZO is the database I/O. This Section therefore discusses the database management of the merged version of BAZO, and what improvements could be made to increase the performance.

BAZO implements a hybrid structure for storing its transactions. While open transactions are stored in RAM, closed transactions are stored in a persistent database [30]. This has two evident advantages:

1. Nodes don't need to store everything in RAM, preventing RAM overflow.
2. In case of a crash, the closed transactions can be retrieved again.

An obvious drawback of this method is that operations on the database are much slower than operations directly on RAM. This means that in postvalidation, deleting from the open transaction pool is very fast, while writing to the closed transaction pool is slow. An easy solution to make the system faster would have been to move all transactions to RAM, but the consequences for BAZO would have been drastic. Instead, realistic measures on how to accelerate database operations were evaluated. Those include database sharding and changing the local structure of how the database was organized at the respective nodes themselves.

After thorough examination of the database management in the merged version of BAZO, it was revealed that writing closed transactions to storage is done in a very inefficient manner. In fact, every transaction is written individually, each time newly accessing the database. This process is on a high level described by Figure 4.5.

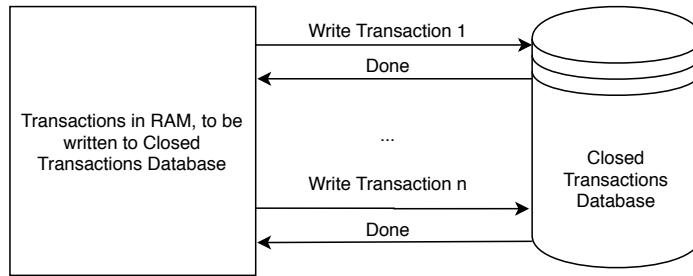


Figure 4.5: Process of writing closed Transactions to Database

For each database call, there is a communication overhead with the database [9]. Obviously, there is no reason why each transaction should be written to the database one-by-one, so methods were implemented to batch calls to the database. In the scope of this thesis, this means that for a pool of transactions to be written to database, all those transactions would be written during one update call at once. This is in the code achieved by first writing all transactions that have to be written to closed storage from an aggregated transaction into a slice, and then sending that slice to Algorithm 6.

From the pseudocode in Algorithm 6, it can be seen that all database I/O operations are indeed done within one *update* statement. This mechanism can be graphically represented by Figure 4.6.

Unsurprisingly, the implementation using batched database calls was able to write closed transactions to the database at a much higher rate than before. In a local testbed, it was measured that the writing process could be done at around 5000 transactions per second, compared to the roughly 350 transactions per second without batching.

Algorithm 6: Batching Database Calls:

WriteClosedFundsTxFromAggTxSlice([]Transaction *transactions*)

Input : []Transaction *transactions*
Output: Error *err*

```

[1] ...
[2] err := db.update(func(tx *bolt.Tx) error) {
[3]   b := tx.Bucket([]byte(bucket))
[4]   forall tx in transactions do
[5]     | hash := tx.Hash()
[6]     | err = b.Put(hash[:], tx.Encode())
[7]     | ...
[8]   end
[9] }
[10] return err

```

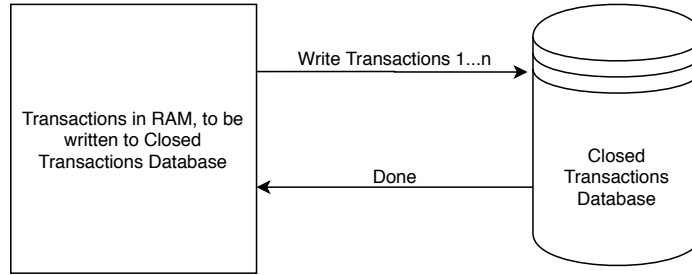


Figure 4.6: Process of writing closed Transactions to Database using Batching

4.2.2 Enhancing the Synchronization Mechanism

Another weakness exhibited by the merged version of BAZO is the mechanism how the shards synchronize their state. To understand the underlying problem, it is important to understand that due to various reasons, it rarely happens that all *validators* mine a block at the same time.

In that case, the first *validator* to successfully mine a block starts requesting the missing state transitions one by one. During the request phase, however, the received state transition stash is never checked. Therefore, it often happens that *validators* request state transitions that they already have in stash. This is unnecessary overhead which frequently slowed down the synchronization process in the testruns by a couple of seconds. It's also worth noting that the problem only gets more severe when more *validators* are added to the blockchain, so a fix to the problem was implemented. Algorithm 7 proposes a solution for the aforementioned problem. Instead of requesting the supposedly missing state transition, the stash is checked again in Line 4. This means, that if the state transition has indeed been received, it won't be requested anymore, mitigating the risk of unnecessarily losing several seconds during the synchronization process.

Algorithm 7: Improved Request Structure in State Synchronization:
 EpochMining([32]byte *hashPrevBlock*, int *heightPrevBlock*)

Input : [32]byte *hashPrevBlock*, int *heightPrevBlock*

Output: none

```

[1] ...
[2] forall id in shardIDs do
[3]   if id != storage.ThisShardID and id not already processed then
[4]     if stateTransition st with st.ShardID == id in stash then
[5]       | continue
[6]     end
[7]     p2p.StateTransitionReqShard(id, lastBlock.height)
[8]     if stateTransition st found then
[9]       | ...
[10]      | apply st on local state
[11]      | set st processed
[12]      | ...
[13]     else
[14]       | wait for 2 seconds
[15]     end
[16]   ...
[17] end
[18] ...
[19] end

```

4.2.3 Evaluation of the Improvements

After implementing the aforementioned performance improvements, BAZO was tested again. For the testbed, the same implementation as in Section 4.1.4 was used. Due to testbed limitations, the improvements were only benchmarked on a 1 shard - 1 validator and on a 2 shards - 2 validators case. Testing with more shards would only make sense once the TPS_{sent} could be improved significantly, as it will become clear from the results.

1 Shard - 1 Validator

Again, the 1 shard - 1 validator case shall serve as a benchmark and show how the sharding improves the performance of the merged version of BAZO. Moreover, since the the the most drastic improvements were made on the database side, the biggest performance jumps can be expected in this configuration. The results can be found in Table 4.6.

Table 4.6: *TPS* with 1 *Validator* in the improved Version

Block Size Bytes	Transactions			
	#sent	#validated	TPS _{sent}	TPS
800	90'000	90'000	1666.7	1363.2

Inspecting the results, two things catch the attention:

1. The TPS_{sent} increased drastically.
2. The TPS increased drastically.

Point 1 could potentially be explained by a lower database load at the receiving node. Since transactions can be handled more quickly, they can also be received faster, because the overall load on the database is lower due to the database improvements. Note that receiving a message includes checking closed storage and open storage if it's already in the system, and then writing it to the open transactions pool. This, however, is only speculation and further tests need to be conducted to find the real reason behind this behaviour.

Point 2, however, brings the desired result. A TPS rate of 1363.2 lies well within the desired rate of 1000-2000 defined as the initial goal of the thesis. This gives a hint that the improvements were done in the right places and put the work on track to reaching the declared goals.

2 Shards - 2 Validators

The results for the 2 shards - 2 validators case with all the improvements can be found in Table 4.7.

Table 4.7: *TPS* with 2 *Validators* in the improved Version

Block Size Bytes	Transactions			
	#sent	#validated	TPS _{sent}	TPS
800	90'000	90'000	1216.2	1034.5

Unfortunately, it becomes clear at this point that the performance limit of BAZO has been reached again. Now, the bottleneck is not how fast transactions can be written to closed storage, but how fast transactions can actually be received by the blockchain, that is the TPS_{sent}. Based on this understanding, it becomes obvious why an evaluation with 3 shards is pointless with the current configuration and testbed.

A possible reason as to why the sending rate is that much higher in Table 4.6 than in Table 4.7 is that in a 2-Shard configuration, the database has a higher stress, and therefore is unable to receive the transactions as quickly as before. There are more separate calls

to the database, mostly owed to the transaction handling process being distributed over more steps, which include handling the own transactions, and later also the transactions from other *validators*. Those conjectures, however, won't be investigated further in the scope of this thesis and merely serve as a starting point for further research on BAZO.

4.2.4 Testing Conclusions

Even though the testing for this part is only small, due to testbed limitations, it yields important insights. Both the 1 shard - 1 validator and the 2 shards - 2 validators case showed drastic performance improvements compared to the results in Section 4.1.6. However, there again was a TPS drop from the 1 shard - 1 validator case to the 2 shards - 2 validators case, which can be explained by both the drop in TPS_{sent} and the transaction management overhead which was already discovered in Section 4.1.6. Still, the testing results, combined with the results from Section 4.1.6 provide enough insights to start with the next step, as the implemented improvements clearly increased the scalability of the system.

4.3 Conclusions from the Scalability Improvement

First of all, even though some progress has already been made, it shall be noted that there is a problem with the testbed that still has to be solved in order to be able to receive convincing testing results. That is, it has to be extended to reach a higher TPS_{sent} in order to show the true performance of the system if more miners are added, and it has to be deployed online.

Still, the test results were able to provide some valuable insights. The two problems that were tackled, namely database I/O speed and communication overhead, had a strong impact together on the system. It was therefore reasonable to shift the focus on the remaining problems that BAZO exhibited at that stage. In fact, two problems which were discovered in Section 4.1.6 haven't been touched yet. Those are load balancing and more importantly the unnecessary replication of the same task, namely the *transaction management* of transactions which were validated in other *shards*. If not fixed, the *transaction management* overhead for a fixed *validator* will increase linearly with the number of *validators* in the system. This overhead contradicts the very principle of *Sharding*.

Chapter 5

Committee Design

In this phase of the thesis, a step was taken back to evaluate the current state not only according to its performance, but also with respect to security and the reliability when deploying online.

On one hand, there were the open issues discussed in Section 4.3 which have to be addressed in order to reach a truly scalable version of BAZO. On the other hand, it became more and more clear that the trustworthiness and reliability of BAZO in its current state left too much to be desired.

In order to fix as many problems which BAZO exhibited at that moment, some drastic changes had to be taken, completely changing the way in which BAZO works. The following Chapter therefore proposes a solution to the problems which the current implementation of BAZO exhibits. From now on, the proposed version of BAZO will be called the *committee version*.

5.1 Motivation

First of all, a major obstacle to running the blockchain in a truly distributed way again is the missing rebroadcasting of transactions to the system, a measure taken in Chapter 4. A new version of BAZO therefore should find a consensus mechanism which doesn't rely on the rebroadcasting of transactions in an effort to minimize the network congestion.

Moreover, in the *merged version* of BAZO, there is currently no robust validation process in place, due to the limitations of the implemented sharding mechanism [11]. This allows for a multiplicity of possible attack scenarios, the most serious of them being a 51% attack. No blocks that the *validators* mine are validated by other nodes, so they could include all kinds of forged information which would be spread across the network.

To address those main issues, this Chapter therefore proposes the addition of a trusted entity called *committee*. It does not participate in the mining process, but it assigns transactions to the shards and performs the necessary validation and authentication steps

to make BAZO a more trustworthy blockchain. In fact, the committee can be extended to contain more than one node, but this does not change its role in any way. Therefore, when something is referred to as "*the committee*", it means the whole pool of members inside the committee.

Another aspect which hasn't been done justice is *authentication*. For example, transaction assignments are not signed by the sender, so they could be faked by another, unauthorized miner. A more convincing version of BAZO will therefore eliminate the possibilities of such attacks at both the committee as well as in the *shards*.

Moreover, *validators* should only be provided with the transactions that they are supposed to validate at a particular step in the blockchain, in an effort to address the *transaction management* overhead issues which were observed previously. They shouldn't have their own transactions database, and most importantly, they should never get in contact with transactions which are handled by other *validators*. Firstly, this makes the *Sharding* mechanism more pure, because there is less redundancy at the *shards*. Moreover, transaction rebroadcasts aren't necessary anymore, because the *validators* are already provided with exactly the transactions that they need. A global view of transactions inside the *shards* is therefore obsolete.

5.2 Assumptions

The main use case for the *committee version* of BAZO is to enable IoT use cases where data has to be processed and stored in a reliable manner. Scenarios include smart factories, smart farming and smart supply chains. All design choices are taken with this in mind.

Unfortunately, the proposed *committee version* has to make some assumptions about the system in order to keep the complexity in the area of a Bachelor Thesis. However, if some of those assumptions prove to be unrealistic in a certain scenario, they can be eliminated by extending the system in a later development step. The assumptions are as follows:

One Validator per Shard

First of all, just like in the sharded version of BAZO, only one *validator* can be added to a shard [11]. This is because previous attempts to add more *validators* to a shard haven't shown any satisfying results. For this reason, the following Chapters may use the terms *validator* and *shard* interchangeably.

Validators have to be running

This limitation was also taken over from the sharded version of BAZO [11]. Since every staking *validator* in the system is assigned to a partition of the global transaction pool, the synchronization mechanism relies on a response from all *validators*, which will become clear in the next Sections of this Chapter.

Committee has to be running

Just like the *validators*, the *committee* needs to be running at all times. If the committee is not running, then no transactions can be assigned and no blocks can be validated. In short, all staking *validators* and all *committee* members need to be online at all times, or the system gets stuck until the connection is re-established.

Epoch Length = 1

In order to keep the new synchronization mechanism at a reasonable complexity, it was decided to fix the epoch length at 1. This is due to the global state only being available in the *leader* as well as in the epoch block. Having longer epochs is possible, but it would mean to either add back more inter-shard communication or reducing the security of the blockchain because *validators* are missing information about the blockchain.

5.3 Entity Management & Assignment

The *committee version* of BAZO has a completely new entity management compared to the sharded version of BAZO [11]. This is mainly due to the addition of the *committee*. *Users* and *validators* stay mostly unchanged and were taken over from the sharded version of BAZO [11]. However, the *validators*' responsibilities were cut off in order to make the system more scalable. They no longer have to keep track of the global transaction pool, but only focus on the transactions that were assigned to them. This means, that they no longer have to fetch, store and delete transactions that they never validated in the first place. However, their responsibilities were extended to always having the right to mine a block, because there is only one *validator* per shard and thus there is no competition or sortition for block creation.

Meanwhile, the *shard leader* changed its role and is responsible for mining the epoch block. Therefore, there is only one *shard leader* in the entire blockchain. In an effort to reduce inter-shard communication, the *shard leader* keeps track of the relative state changes that the other *validators* have to provide and summarizes those in a global state that is written inside the epoch block. Other *validators* can then read the global state directly from the epoch block, thus simplifying the inter-shard communication considerably. Moreover, the *leader* is a subset of all *validators* and it is also responsible for validating transactions and mining blocks. The *leader* for a certain epoch n gets elected in the randomized validators to shard assignment mechanism in the previous epoch $n - 1$.

As a completely separate entity pool, there is the *committee*. It can be seen as the owner of the blockchain and ensures its integrity. Inside the committee, there are two kinds of entities: committee *members* and a committee *leader*. As it is the case in the shards, the leader is a subset of the members. That is, the leader for a certain epoch n gets elected in the randomized *leader* assignment mechanism in the previous epoch $n - 1$.

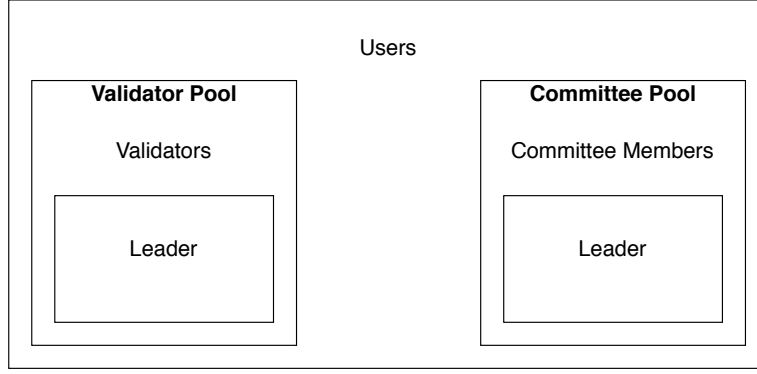


Figure 5.1: Entities in BAZO

The *committee leader* is responsible of assigning transactions to the shards, and also to run the byzantine consensus mechanism to come to a slashing agreement if malicious behaviour could be detected in the blockchain. Meanwhile, all committee *members* validate the blockchain using a mechanism which will be introduced in a later Section. In order not to leave the *committee leader* of a certain height with too much power, the *committee leader* of epoch n is responsible for the byzantine consensus mechanism of epoch $n - 1$.

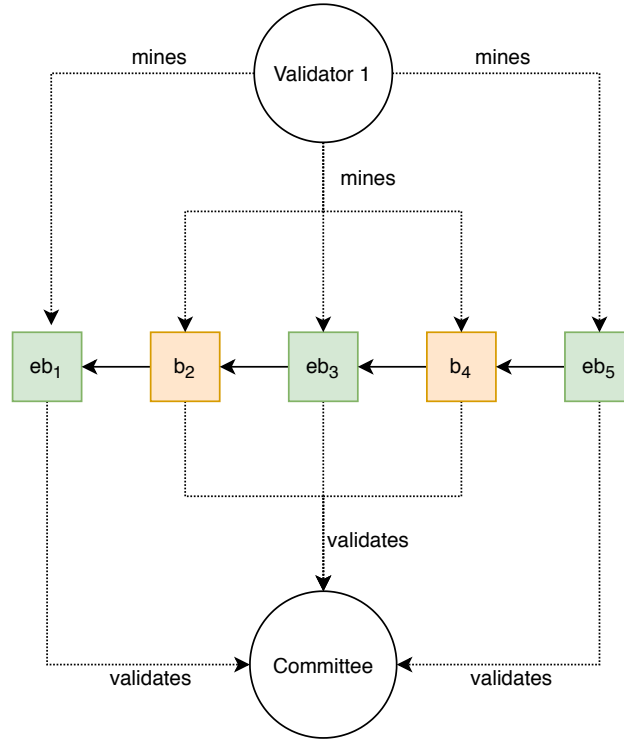


Figure 5.2: Schematic Representation of the Validation Process

Figure 5.1 shows the two entity type pools and how they relate to each other. Firstly, all nodes in the blockchain have a wallet, which makes them a *user*. They either stay normal users, or they join one of the two pools. If the user joins the **Validator Pool**, he automatically becomes a *validator*, with the chance of becoming the *leader* in every epoch. If, however, the user joins the **Committee Pool**, he automatically becomes a *committee member*, with the chance of becoming the *leader* in every epoch.

In order to understand the difference between the shards and the committee, a schematic representation of the validation process can be found in Figure 5.2. For simplicity, a one-shard one-committee configuration was chosen. In the middle, the colored blocks represent the blockchain. The blocks labelled with *eb* represent epoch blocks, while the blocks labelled with *b* represent normal blocks. The subscript represents the block height. The circles represent the nodes inside the blockchain, on the top the *validator*, and on the bottom the *committee*.

As it can be seen, the responsibilities are clearly distributed. While the *validator* only mines blocks, the *committee* only validates them. Since the *committee* is a trusted entity, after blocks were confirmed valid by it, they can safely remain in the blockchain.

5.4 Data Transactions

Since the main focus of the *committee version* of BAZO was to securely and reliably process data streams of IoT devices, it became necessary to add an additional type of transaction to the system. The data transaction can be used to send streams of data from one address to another and store this transaction inside the blockchain. Since the data inside those transactions doesn't change the blockchain in any way, it was not necessary to change the blockchain in a drastic way in order to accomodate it. Nevertheless, data transactions offer a valuable addition to the system, which will be explained in the following Sections.

Data transactions follow the same principles as other transaction types in BAZO, with the most similar transaction type being the funds transaction, since both for the most part have the same fields, and both can be aggregated. They have to be signed using the Elliptic Curve Digital Signature Algorithm, also abbreviated as ECDSA, using both the private wallet key from the sender as well as from the root account.

The data that is sent inside data transactions can have many different forms. While one sender might decide to send strings, another sender may opt to send serialized Java objects. By forcing all clients to send the unified format of bytestreams, they can all be stored in the same way with no extra overhead at the blockchain's side. When the information is retrieved again, the bytestream can be augmented with the information about the original data format, and the original data to be stored can be recreated again.

In order to keep the scalability improvements that *Transaction Aggregation* offered BAZO [27], it is also possible to aggregate any amount of data transactions to an aggregated data transaction if they all have the same sender. The main benefit of this process is to save memory in the blockchain by requiring to store only one aggregated transaction instead of all transactions individually. More details of how this mechanism will work when both FundsTx and DataTx are available to be aggregated are discussed in Section 5.5.

5.5 Transaction Aggregation

In Section 5.4, it was mentioned that the *validators* perform a transaction aggregation mechanism using both FundsTx and DataTx, resulting in aggregated FundsTx and aggregated DataTx respectively. This mechanism is in many ways similar to the one found in the merged version of BAZO, with some decisive differences. The most severe of those is the addition of a new transaction which is also able to be aggregated, namely the DataTx, which means that two types of transactions are competing against each other to be added to the block. In order to aggregate as many transactions as possible, and thereby benefiting the most from the scalability benefits of the transaction aggregation mechanism, BAZO aggregates the transactions such that the maximal amount of FundsTx or DataTx can be added to the block. If a set of FundsTx and another set of DataTx to be aggregated have the same size, the FundsTx will be given priority as it bears a more important immediate impact on the blockchain. For more details about the exact implementation of the Transaction Aggregation mechanism, Fabio Maddaloni's Bachelor Thesis can be consulted [27].

5.6 Transaction Management & Assignment

Since the *validators* don't have a global view of the transactions anymore, they rely on the *committee* to supply them with a portion of the total transaction pool with transactions that have not been validated yet.

At the beginning of each epoch, the *committee* evaluates the number of *validators* in the system and assigns to each of them a partition of the pool of open transactions according to the public address of the transactions' sender. In order to do that, a new data type called transaction assignment has been introduced to BAZO. In short, it includes all transactions from the global transaction pool that have been assigned to a specific *validator* at the time of the creation of the assignment. The assignment mechanism is similar to the one in the sharded version of BAZO [11], but the responsibility for the transaction sharding has been shifted to the *committee*. This avoids the multiple execution of the same task in each shard and effectively makes the *validators* more light-weight.

After a block is mined, containing an ideal set of transactions according to the Transaction Aggregation Mechanism described in Section 5.5, it is broadcast to the network so it can be validated by the *committee*. Moreover, the state transition is sent out to the entire network for synchronization and validation purposes.

On the *validator* side, the transaction assignment and management mechanism can be simplified and graphically represented like in Figure 5.3. From the assignment, the local memory pool is filled up with transactions. Those transactions are represented by a green dot. The *validator* then chooses the optimal combination of transactions to add to the block, the chosen ones staying green, and the ignored ones being colored in red. After the block creation, both the transactions that were validated and the ones that weren't validated are completely deleted from the memory pool.

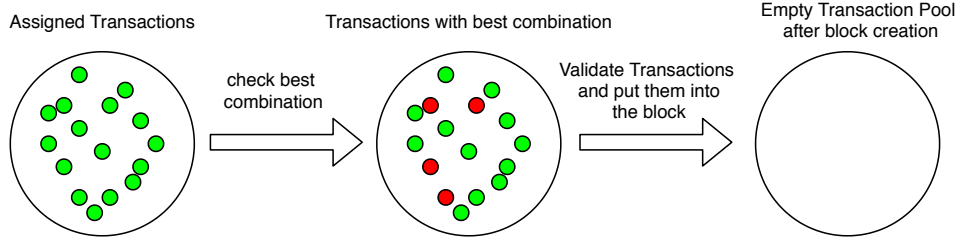


Figure 5.3: Schematic Representation of the Transaction Management from the *Validator's* Perspective

The *committee*, upon reception of the block from the *validators*, fetches all transactions which were included in the block and writes them to the closed transactions pool. The same set of transactions is then also deleted from the open transaction pool. Simplified in a similar manner as the *validator* transaction assignment and management view, the *committee's* view can be represented in Figure 5.4, if considering a 3 shard - 1 committee configuration.

First, the global open transaction pool is loaded into memory, each transaction being represented by a green dot. Then the whole pool is partitioned into 3 different partitions, each representing one shard, according to the partition mechanism. Those partitions are then sent out to the *validators*. The transactions validated inside the block are then deleted from the mempool, marking them red. The remaining transactions stay green and are sent out to the assigned *validators* again in the next step.

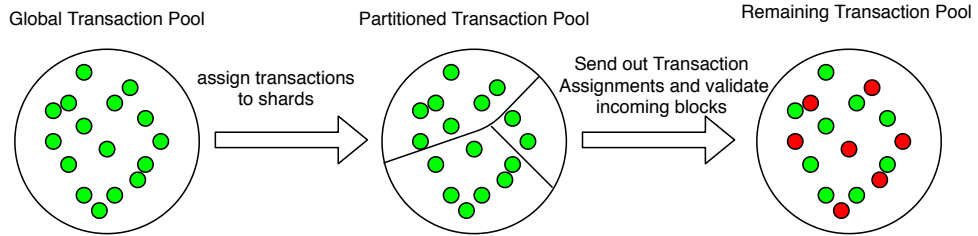


Figure 5.4: Schematic Representation of the Transaction Management from the *Committee's* Perspective, in a 3 Shard Configuration

By keeping track of all transactions which were not validated by the *validators*, the *committee* does not have to force the *validators* to validate all transactions that they were assigned to. If there are leftover transactions that were not validated, they will be put into the transaction assignment mechanism at the next block height again, and they get another chance to be validated by the system.

5.7 Communication & Synchronization

As it has been mentioned before, one of the main design choices was to keep the communication overhead as low as possible. In order to reach this goal, the previous synchronization mechanism from the sharded version of BAZO [11] has been made more light-weight by

reducing the inter-shard communication overhead, and the added communication between the *committee* and the *validators* has been designed to be as minimal as possible. In the following Section, the synchronization and communication protocols used in the *committee version* of BAZO will be presented in detail.

5.7.1 Motivation

The reason why there has to be inter-shard synchronization has already been discussed in Kürsat Aydinli's Master thesis [11]. Additionally, the validator-committee synchronization is necessary to keep a global view of the open transactions as well as to decide which *validator* validates which transactions at a certain height. If there were no synchronization, malicious behaviour such as double spending might be possible.

5.7.2 Inter-Shard Communication & Synchronization

In the inter-shard communication, it can be seen what benefits a fixed epoch length of 1 brings to the system. In the sharded version of Bazo [11], the task of aggregating state transitions to a global state was distributed to all shards, meaning that the same task was done by all *validators*. Also, all state transitions had to be sent to all other *validators*. If we denote the total amount of *validators* as n , this leads to a total of $\frac{n*(n-1)}{2}$ connections that have to be active. Moreover, by the same logic, a total of $n * (n - 1)$ messages have to be sent around the network at each block height. This leads to a complexity of $\mathcal{O}(n^2)$ messages in the system at each block height, which was something to be avoided for the *committee version* of BAZO.

In order to solve this problem, the epoch length was fixed at 1. State transitions are created and distributed by all *validators*, and they are applied and aggregated by the *leader* in a similar manner as in the sharded version of BAZO [11] before creating the epoch block. Obviously, the last step of deleting the transactions that were validated by the other *validators* is omitted, because the *leader* does not have those transactions in the memory pool in the first place. Note that since only the leader has to receive the state transitions, the complexity of messages for each block height has been reduced to $\mathcal{O}(n)$.

This mechanism guarantees that the shard *leader* is up to date after receiving all state transitions. Exactly for this reason, it is the *leader's* task to create the epoch block with the updated global state in it, such that the other shards and the committee synchronize the global state as well, along with the shard and committee leader assignment.

5.7.3 Inter-Committee Communication & Synchronization

Just like the shards, the committee members need to synchronize with each other. The reason for this is that for each step, it is essential that they work with the same data, and that they keep their memory pools synchronized. In fact, it should be irrelevant to the

shards how many members the committee counts, because the committee should act as a synchronized and unified entity.

One major goal of inter-committee communication is to enable the committee to decentralize power by running the validation task not only in one node, but in all committee nodes in parallel, and to come to a unified and definitive decision in the end. In order to achieve that, the *committee members* all receive the transaction assignment from the committee *leader*, such that every *member* has the same data to work with while validating. During the validation process, each committee node performs exactly the same steps using the same data, which will, in the end, help them find a consensus of which nodes acted maliciously.

After the validation process, the *committee members* need to synchronize in order to find consensus about which nodes acted maliciously. This is reached by each *committee member* sending out a committee check, containing information about which nodes they identified as being malicious. Again, it is only the *leader* who has to process those messages.

Just like in the inter-shard communication, it was a major concern to keep the total amount of required messages for the inter-committee communication as low as possible: It is only the *leader* who sends out the transaction assignment out to the other *members*, and it is only the *leader* who receives the committee check messages from the other *members*, leading to a complexity of $\mathcal{O}(n)$ of sent messages, as opposed to $\mathcal{O}(n^2)$, with n denoting the number of nodes in the system.

5.7.4 Validator-Committee Communication & Synchronization

Now that the communication between the *validators* and inside the *committee* is regulated, those two entities still need to synchronize and communicate with each other. In short, the communication can be grouped into two touchpoints:

1. Sending out transaction assignments
2. Validating blocks and the epoch block

At each block height, the *validators* have to wait for the transaction assignment from the *committee*. The reception of the transaction assignment then serves as the starting point for mining a new block.

Meanwhile, once the *committee* received and validated all blocks, it can build the new global open transaction pool with the remaining, not validated transactions, and after the reception of the epoch block with its validator shard assignment, it is ready to partition the global open transaction pool according to the *validators* again. Note that an epoch block with height h can only arrive at the committee if all blocks in epoch with height h have been mined, and, more importantly, after all transaction assignments have been received.

The synchronization that happens at the aforementioned touchpoints guarantees that the *validators* and the *committee* are always at the same block height, making problems such as the ones discovered in Chapter 3 impossible. A schematic representation of how the communication mechanism between the *validators* and the *committee* works can be found in Figure 5.5. It shows a simplified version of a state machine for the *committee* on the top half as well as a simplified state machine for the *validators* in the shards on the bottom half. A transition to the next state is depicted by a solid arrow, while messages are represented by dashed arrows. Whenever there is an incoming message, the machine halts until all required incoming messages have arrived.

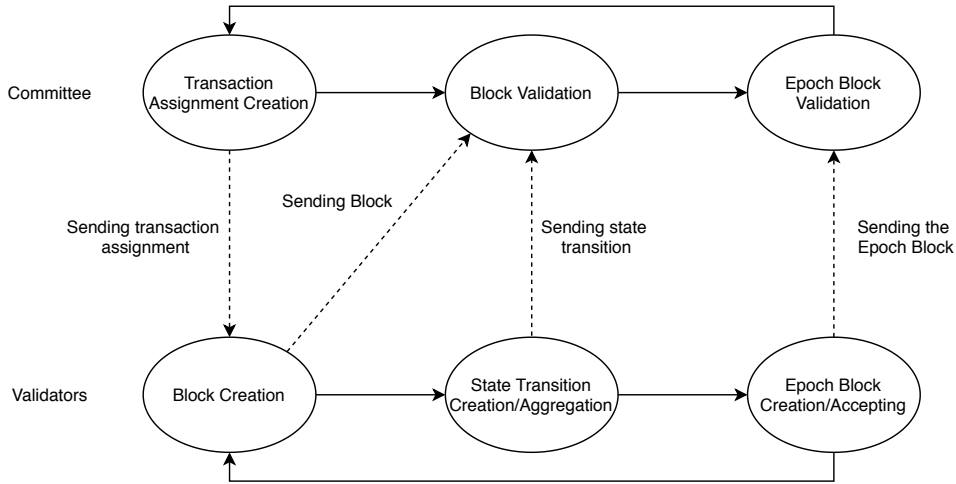


Figure 5.5: Schematic Representation of the Communication Mechanism

Effectively, having those touchpoints at both state machines enables them to be synchronized. For example, by construction, it is not possible to send two consecutive transaction assignments before a block for the first transaction assignment has been mined and sent to the network.

Another feature of the validator-committee communication is its use for updating the closed memory pool. Since all committee members validate all blocks, they can all write the transactions which were contained in the blocks to the closed memory pool. Furthermore, to synchronize the global state of the *validators* with the global state of the committee, and to communicate the new shard and committee leader assignments, all committee members also validate the epoch blocks.

5.8 Authentication & Validation

Authentication and Validation were major concerns during the design of the *committee* version of BAZO. In the sharded version of BAZO, which served as the initial sharding and load-balancing implementation for this work, there was no validation in place whatsoever [11], along with insufficient authentication. Until now, the strategy which should make BAZO more reliable haven't been discussed, which will be rectified in this Section.

5.8.1 Authentication Strategy

The first problem that got tackled was the inter-shard communication. The protocol used for inter-shard communications is called *state transition* and only implements naive authentication mechanisms. That is, for each *state transition*, the shard ID and the block height of the block where it originated from were passed to the other shards. Of course, it would be easy for malicious shards, or even outsiders, to create such a state transition in another shard's name, because there is no proper identity check. To combat this issue, *state transitions*, just like blocks and epoch blocks have to be signed using their private commitment key using an RSA signature mechanism. Using the public commitment key, which is publicly available in the blockchain state, the validity of the key can be checked.

In a similar fashion, the inter-committee communication and the shard-committee communication was extended by signatures and checks, such that no malicious node inside or outside the blockchain can assume a wrong identity. Therefore, the **authenticity** problem of communications was solved.

5.8.2 Validation Strategy

At this point, however, only a part of the problems are solved. All communication channels are authenticated, which means that all messages where the signature can be verified can be trusted regarding to the sender. Unfortunately, malicious behaviour by the shards is still possible. Only if the **validity** and **authenticity** of all communications are given, can the blockchain be trusted. To understand the problem, it is necessary to know what state transitions were used for in the first place: The problem that the *sharded version* of BAZO was facing was that even in a sharded blockchain, the global state has to be somehow maintained in order to reliably be able to validate transactions. However, letting all shards validate all transactions would defeat the purpose of having a sharding mechanism in the first place. Therefore, it was decided that shards communicate with each other using state transitions, which only carry information about how the state has to be changed, without any further checks in place [11].

At this point, the committee comes into play. It checks whether they included any transactions in their block that they were not assigned to, and if the state transitions that they produce correspond to the actually validated transactions. In order to do that, the committee creates its own state transition, based on the transactions that were put inside the block, and compares it to the state transition which was produced by the shard itself. The check for validity of the state transition is crucial, because without checks, any numbers could be included in the relative state, which would remain undetected in the blockchain. This is because the block creation and state transition creation processes are decoupled: Even if the produced block is completely valid, it is possible that the state transition contains bogus information.

Next, the problem concerning the blockchain state consolidation at the epoch block has to be addressed. Just like in the *sharded version* of BAZO, it is the task of a single shard, in the committee version called the *shard leader*, to aggregate all state transitions with

the own state and consolidate it in the epoch block [11]. All other nodes in the network then accept the global state from that epoch block. The **validity** and **authenticity** of the epoch block are therefore crucial to a trustworthy system. Fortunately, **authenticity** was already addressed by previous implementations, but again, the **validity** had to be checked. This is done by aggregating all relative states from all shards in the system, creating an aggregated relative state and comparing that relative state to the difference between the epoch block being checked and the epoch block of the previous height. The remaining validation mechanisms are implemented in a similar fashion, which can be observed in Chapter 6.

5.9 Assessment of the Design

In Section 2.3, several alternative blockchains have been presented. The goal of this section is to compare those projects with each other as well as the *committee version* of BAZO.

5.9.1 Architecture

In this Section, a short comparison of the architecture of the projects is presented.

BAZO

The architecture features of the *committee version* of BAZO are covered in detail in Chapter 5. In a nutshell, BAZO consists of two main entity types, the *shards* and the *committee*. The *Sharding* mechanism allows for dynamic load balancing, which means that for each added *shard*, the transaction pool is partitioned into one additional partition. Since each *shard* gets assigned to exactly one partition, increasing the number of *shards* leads to a smaller workload in each individual *shard*.

The *committee*, on the other hand, is in charge of validating the blocks which are produced by the *shards*, along with the contained transactions. Just like the *shards*, the *committee* can grow dynamically. With each added *committee member*, there is an additional layer of security and a malicious takeover of the blockchain requires a higher amount of compromised nodes in order to be successful.

Zilliqa

Just like BAZO, Zilliqa implements a mechanism consisting of two entity types. There are the nodes in the shards, and the nodes in the DS Committee. Contrary to BAZO, the nodes are shifted around those entities after every epoch. Moreover, a new shard is only introduced for each 600 nodes in the network. For every added shard, the scalability improvement is linear [19].

RapidChain

Just like the previous projects, RapidChain implements sharding. During bootstrap, a small amount of the total nodes are elected to be the reference Committee. The reference Committee is then in charge of partitioning the validator pool onto the shards [33].

QuarkChain

QuarkChain has arguably the most similar architecture to BAZO. It is designed using two layers, one which is in charge of mining blocks and one which is in charge of validating them [14].

OmniLedger

OmniLedger consists of two different blockchain systems: The actual blockchain where transactions are validated, and an Identity Blockchain to verify the identity of the nodes in the network. Inside the blockchain, there are two types of entities: The Optimistic Validators and the Core Validators. The Optimistic Validators mine blocks with only rudimentary validation, and the Core Validators confirm those blocks [23].

Libra

Libra follows a more traditional architecture strategy compared to the previously described projects. It is a traditional blockchain, and with each added node, the performance decreases whereas the security is increased [26] [34].

5.9.2 Use Cases

This Section will compare the intended use cases for the presented blockchains.

BAZO

BAZO is intended to be used for global IoT systems.

Zilliqa

Since Zilliqa focuses on high performance and supports smart contracts, its main use case is for building decentralized applications [10].

RapidChain

RapidChain mainly positions itself as a rival to Bitcoin. It aims to improve the poor transaction rate which Bitcoin achieves at the moment [33].

QuarkChain

QuarkChain aims to put itself into direct competition with Bitcoin and Ethereum. It wants to support the needs of worldwide financial markets, and by the adaption of the Ethereum Virtual Machine (EVM) it supports the migration of existing Ethereum projects to QuarkChain [6].

OmniLedger

OmniLedger's main goal is to increase the TPS of traditional blockchains in order to rival traditional centralized payment processors [23].

Libra

Libra aims to create its own currency and payment infrastructure using blockchain technologies [5].

5.9.3 Permissions & Coverage

This Section will compare the permission systems and the coverage of the presented blockchains.

BAZO

BAZO is a permissioned blockchain, and if a new node wants to join the network, it needs the private wallet key of the root node [30]. It is, however, designed for global use.

Zilliqa

Zilliqa is a permissionless blockchain and already in global use [1].

RapidChain

Rapidchain is designed to be permissionless and run in a global system in an attempt to rival global financial systems [33].

QuarkChain

QuarkChain is a permissionless blockchain and already in global use [14].

OmniLedger

Omniledger is designed to be permissionless [23].

Libra

For its launch, Libra is designed to be permissioned. However, in the future, a permissionless design is planned [17].

5.9.4 Scalability

This section discusses the scalability of the the selected blockchain technologies.

BAZO

BAZO, in ideal conditions, scales linearly with the addition of more shards. The complete evaluation can be found in Chapter 7.

Zilliqa

Just like BAZO, Zilliqa scales linearly with the addition of more shards. However, one shard requires 600 nodes [19].

RapidChain

By parallelizing the consensus work, RapidChain claims to scale linearly with each added node [33].

QuarkChain

It is claimed that by adding more shards to the network, at the moment the system can support a theoretical 100'000 TPS [7].

OmniLedger

OmniLedger is claimed to scale linearly with each added node [23].

Libra

In testing it was shown that Libra in its current state gets slower with each added node [34].

Chapter 6

Implementation

In this Chapter, the technical implementation of the design proposed in the last chapter is discussed.

6.1 Consensus Mechanism

As it has become clear in Chapter 5, the *committee* version of BAZO implements a completely new system to reach consensus compared to the sharded version of BAZO [11]. The goal of this Chapter is to present how this mechanism was implemented on top of the existing BAZO project.

6.1.1 Consensus in the Shards

The *shards*' role for the consensus mechanism is crucial. It's the *shards* who validate transactions and add them to the global state as a first step of the consensus mechanism as a whole. The pool of *shards* can be described by two different entities with different tasks, as it will be described by the following Sections.

Shard Leader

There is a multiplicity of different ways how the *shard leader* could be elected. Following a pragmatic approach, however, it was decided to implement the consensus mechanism in such a way that shard with *shardID* = 1 should be the *leader* for the respective epoch.

The *leader* has a special role inside the *shard* pool. For better understanding, consider the flowchart inside Figure 6.1. Firstly, before being able to mine a block, the transaction assignment has to arrive. After that, a block is mined. Then, the *leader* waits for all state transitions from the other *shards* and requests missing ones if necessary. For each state transition, it updates its local state by applying the relative state as it has been done in

the *sharded* version of BAZO [11]. After all state transitions have been processed, the *leader* has the updated current global state of the blockchain. As it is the only *shard* node with this global information, it then produces the the epoch block containing the global state in it. Moreover, it produces a new random validator shard assignment and a assigns a new random *committee leader*.

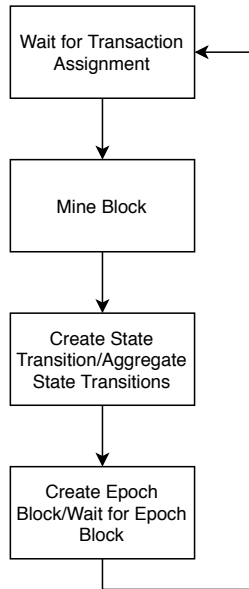


Figure 6.1: Flowchart of the Shards

Other Shards

From the *leader* condition in Section 6.1.1 it directly follows that *other shards* are defined as exactly those shards where $shardID \neq 1$.

To understand the workings of the other *validators*, consider Figure 6.1 again. First, the *validator* has to wait for the transaction assignment. After that it can mine a block and send out the state transition. Then it waits for the epoch block to arrive so it can take over the new global state, along with the new shard assignment. From a *shard's* perspective, consensus is reached at this moment.

Adding more Miners to a Shard

Now that there is a completely new architecture and system, it is the time to go back and evaluate what existing structures could be re-used. In this case, it's the *multiple miners per shard* version proposed in Section 4.1.2.

Unfortunately, however, the new mechanisms of the *committee* version of BAZO make the *multiple miners per shard* mechanisms obsolete. The reason for this is that there is already a minimal amount of communication between the *validators*. In fact, there are no state transitions anymore which have to be reached by all nodes in the network. This drastically

simplifies the synchronization to a point where a grouping into *inter-shard communication* and *intra-shard communication* doesn't promise to make any improvements anymore: Every *validator* just directly reports to the *leader*.

For those reasons, it was decided not to follow this path anymore, instead focusing on implementing more important features that had to be brought to BAZO.

6.1.2 Consensus in the Committee

As it should be clear to the reader, the consensus mechanism inside the *shards* is insufficient to propose a trustworthy system. For this reason, the consensus mechanism was expanded by the addition of the *committee*.

Committee Leader

Just like it is the case with *shard leader* formation, there are many different ways how the *committee leader* could be elected. Following a similar path as it was in the shards, the *committee leader* is elected during epoch block creation in a randomized fashion. Other than the validator shard assignment however, the other *committee* members don't need any additional information, which was necessary for the *shards* to partition the transaction pool, i.e. shard IDs. This is because the *shards* all do a different task, while the *validation* process is the same for all *committee* members. Therefore, the *committee leader* is stored directly inside the epoch block, with no assignment for the *other committee members*.

While the *committee leader* has more responsibilities than the other *committee members*, that is running the byzantine consensus mechanism and assigning transactions to the shards, it does not hold more power than them in the consensus mechanism. Since the work of all *committee members* is very similar, the exact description of the mechanism is deferred to the next part.

Committee Members

In order to understand how consensus at the *committee* is reached, consider Figure 6.2. It explains the workings of the *committee* on a high level.

At the beginning of each round, only the *committee leader* is active. It runs the byzantine consensus mechanism as later described in this Section, slashes nodes that have acted maliciously in the previous round, and creates and sends out the transaction assignments as soon as they are done. While the other *committee members* have to wait to get all transaction assignments before being able to continue, the *committee leader* goes directly to the next step because it already has them. From now on, all *committee members* behave in the same way. That is they start listening to, requesting and authenticating blocks and state transitions in parallel. This is achieved by spawning a separate goroutine from the main algorithm as follows: `go fetchStateTransitionsForHeight(height, &waitGroup)`. The

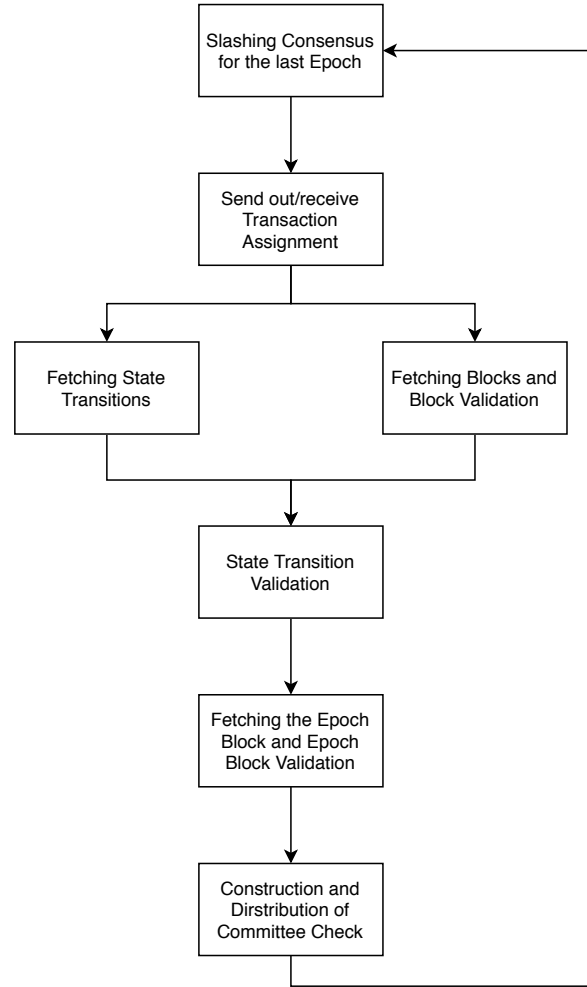


Figure 6.2: Flowchart of the Committee

waitgroup is passed to the goroutine such that the main algorithm can be synchronized again once the goroutine is done. This parallel execution of the code has the advantage that time for fetching the state transitions from the network can be saved, since it is done much faster than the main algorithm. During block validation, the validity of the PoS and of the contained transactions is checked according to the security mechanisms described in Section 6.4.3. Moreover, a relative state is constructed, which is used to validate the state transitions at the next step, which will be entered once both the previous two steps are done. State transitions are then validated according to the validation mechanism in Section 6.4.3. After that, the epoch block is validated according to the mechanisms from Section 6.4.3. Then, the epoch block with its new state and assignments is accepted, and all *committee members* send out their committee checks containing the members who acted maliciously in the round. Those committee checks can then be used by the *leader* in the next round for the byzantine consensus mechanism.

Byzantine Consensus Mechanism

The slashing in the *committee version* of BAZO is regulated through a byzantine consensus mechanism and gives the system a way to combat malicious behaviour. As long as there is no stable rollback mechanism, this is the best way how to handle such situations. However, as it could be seen from the Related Work Sections, there is a push from the industry and academia for blockchain projects with immediate finality, which can be seen at the example of Zilliqa [19], which means that rollback mechanisms might not be a part of scalable blockchains anymore in the future. For this reason, it was decided to not make an effort to build rollbacks back into BAZO.

As it has been mentioned in the previous Subsections, all *committee members* validate the work of all *validators* in parallel. At the end of each round, the *committee members* prepare a summary of all nodes for which they detected malicious behaviour. In order to do that, they create a committee check, containing a slice of all *validator* addresses who acted maliciously as well as a slice for all *committee member* addresses who acted maliciously in the epoch. The message is then signed and broadcast to the network. In the next epoch, the new *committee leader* collects all of those committee checks and based on the complete set of these, performs the byzantine consensus mechanism. The mechanism is based on voting, which means that based on the total amount of *committee members* in the system, a certain amount of those has to vote to slash a specific node in the blockchain. The percentage of *committee members* needed in order to slash a node can be parametrized in the `..\miner\configs.go` file by changing the value in `PERCENTAGE_NEEDED_FOR_SLASHING`. By default, at least 66.67% of the *committee members* have to vote in favor of slashing in order for the slashing to be executed. Algorithm 8 describes the process. First, the number of votes from *committee members* which are required for slashing is evaluated. Then the maps `fineMapShards` and `fineMapCommittees` for counting are instantiated. Those two maps are kept separated such that the system allows for different methods of handling malicious behaviour based on the entity type of the offender. Then, all accounts in the current global state are evaluated in a loop. For each committee check which includes the address of the current account being evaluated, the counter in the respective map is increased by one. After all accounts have been evaluated, the maps are checked for entries which have enough votes to be slashed.

Inside the functions `SlashShard(address)` and `SlashCommittee(address)`, the measures to slash the malicious nodes are implemented. For this implementation of BAZO, a pragmatic slashing mechanism was chosen, which is extendable in future iterations of the project. For now, both those functions create a new transaction, which is called fine transaction. Inside the `..\miner\configs.go` file, the standard fine for malicious shards and malicious committee members can be set by changing the variable of `DEFAULT_FINE_SHARD` and `DEFAULT_FINE_COMMITTEE` respectively. The fine transaction will then contain this number from the file, and will be shared with the other committee members for validation purposes. When the transaction reaches the *validators* through the transaction assignment, and is validated, the specified amount will be deducted from the wallet of the malicious node. Note that the penalties are fixed for this build. This means that no matter what the malicious behaviour was, it will always be punished in the same way. Further iterations of BAZO could include a slashing mechanism which takes the severity of the malicious act into consideration.

Algorithm 8: Runing the Byzantine Consensus Mechanism:
 runByzantineMechanism([] CommitteeCheck *committeChecks*)

Input : [] committeeCheck *committeChecks*

Output: none

```

[1] numOfCommittees := DetNumberOfCommittees()
[2] requiredVotes := DetNumberOfVotersForSlashing(numOfCommittees)
[3] create fineMapShards
[4] create fineMapCommittees
[5] ...
[6] forall acc in storage.State do
[7]   if acc.IsStaking then
[8]     forall cc in committeChecks do
[9]       if containsAddress(cc.SlashedShards, acc) then
[10]        | fineMapShards [acc] += 1
[11]      end
[12]    end
[13]  end
[14]  if acc.IsCommittee then
[15]    forall cc in committeChecks do
[16]      if containsAddress(cc.SlashedCommittees, acc) then
[17]        | fineMapCommittees [acc] += 1
[18]      end
[19]    end
[20]  end
[21] end
[22] forall address, votes in fineMapShards do
[23]   if votes >= requiredVotes then
[24]     | SlashShard(address)
[25]   end
[26] end
[27] forall address, votes in fineMapCommittees do
[28]   if votes >= requiredVotes then
[29]     | SlashCommittee(address)
[30]   end
[31] end

```

6.1.3 Global Consensus

From the sections above, it can already be seen how the *shards* and the *committee* reach consensus. In the ideal case, that is if no node in the network acts maliciously, consensus is reached in the *shards* without active intervention by the *committee*. For all members of the blockchain, the epoch block serves as a way to synchronize the global state. If, however, malicious behaviour is detected by the *committee*, consensus has to be reached taking the slashing mechanism presented in the previous section into consideration, which includes issuing fine transactions.

6.2 Transaction Life Cycle

Compared to the previous versions of BAZO, the transaction life cycle has drastically been changed. In all previous versions of BAZO, each transaction had to reach all nodes in the network. One of the main goals of the new implementation was to combat this overhead. To shed more light on how exactly this was done, and why rebroadcasts really are not necessary anymore, the following Section will show the entire lifespan of an arbitrary transaction.

6.2.1 Sending Transactions into the Network

Previously, all transactions were created in the client application and sent to the bootstrap node. In order to achieve this, the bootstrap node's IP address and port had to be specified in the client's *configuration.json* file. Thereafter, the transactions were broadcast throughout the network, starting from the root node. The rule was that every node broadcast every transaction upon reception unless the received transaction was already in the node's storage. This was necessary because the structure of the blockchain required all nodes to have the same global information at the same time: open transactions, closed transactions, and the global state. The rebroadcasting mechanism lead to a high network-load overhead and can be sketched as described in Figure 6.3

It was by a fair part due to this heavy overhead that the p2p package was brought to its knees in early test runs of BAZO.

To tackle this issue, it is now the client's task to broadcast the transaction to all *committee members* in the network. To achieve this, a new json file called *committees.json* was created, which holds the IP addresses and ports of all committee members. The transactions are then bundled by the committee and sent out to the shards in single big packets, also known as transaction assignments, reducing the amount of individual messages inside the network drastically. This can be illustrated by Figure 6.4. It was this change which allowed for the disabling of transaction rebroadcasts.

A snippet of how the json file now looks like is demonstrated below. As it can be seen, it contains a list of committee IP addresses and ports, which has to be extended whenever a

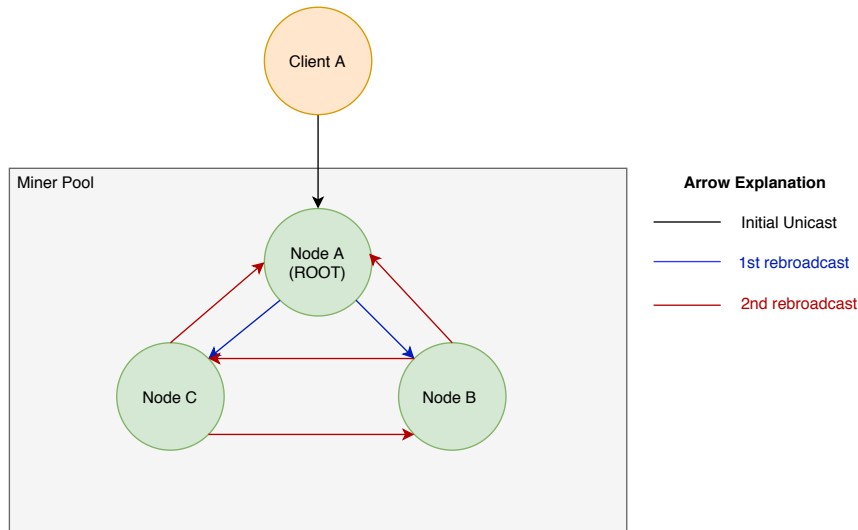


Figure 6.3: Schematic Representation of the Transaction Rebroadcasting

new committee member joins the committee pool. Moreover, it's important to remember that strings of real IPv4 addresses should be used instead of using *localhost* in order to avoid a pitfall such as it was experienced in Section 4.1.1.

```

1 {
2   "committees": [
3     {
4       "ip": "127.0.0.1",
5       "port": "8002"
6     },
7     {
8       "ip": "127.0.0.1",
9       "port": "8003"
10    }
11  ]
12 }
```

6.2.2 Transaction Assignment to the Shards

After the transactions have been broadcast to the committee members as sketched in Figure 6.4, they will be assigned to the shards using a similar mechanism as the one which was proposed in the *sharded* version of BAZO, which means that they will be distributed according to the public address of the sender of the transaction. This is done in the following way: First the entire pool of open transactions is read from memory. Then, for each transaction, its concrete type is evaluated. For each concrete type, there is a map of the form `map[shardID][]concreteTransactionType`. Accordingly, using the function `assignTransactionToShard(transaction)`, the shard ID where the transaction should be

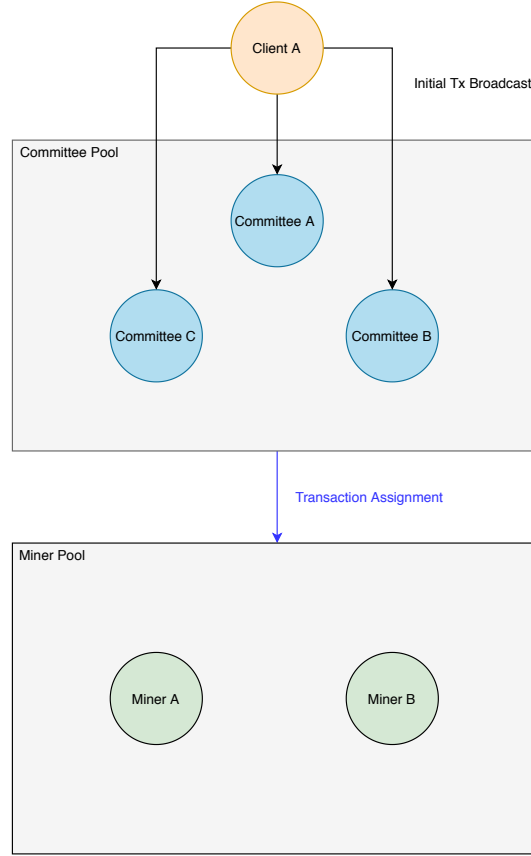


Figure 6.4: Schematic Representation of the new Transaction Lifecycle

assigned is evaluated. This shard ID is then used to access the slice value in the map which corresponds to the given shard ID as the key. Like this, the evaluated transaction can be added to the correct slice as follows:

$$\mathbf{map}[shardID] = \underline{\mathit{append}(\mathbf{map}[shardID], transaction)}$$

After all transactions have been assigned, for each shard ID in the network, a transaction assignment is created and amended by the slices of transactions corresponding to the shard ID, using the shard ID as the key to access the respective transaction slices. After its creation, the transaction assignment is sent to the network, containing the slices of entire transactions, and not just their hashes. Sending entire transactions means that they don't have to be requested separately by the *validators*, minimizing network congestion.

6.2.3 Shard Mining

Upon reception of its verified transaction assignment, the *validator* can start mining the block for its shard. Validators do not hold any persistent storage, but only work with the transactions that they are assigned to. After completing the verification and aggregation steps, the chosen transactions are applied on the local state, the state transition is created

and the block is mined. After that, the *validator* completely forgets about all of the previously received transactions and waits for the next transaction assignment.

6.2.4 Committee Validation

After the reception of a block, the validation mechanism starts. If the transaction was chosen to be included in the block, it will be written to closed storage. If it wasn't included in the block, it will remain in the open transaction pool and it will be assigned again in the next transaction assignment round.

6.3 Data Management & Data Retrieval

Contrary to FundsTx, DataTx don't store their contained information directly to the blockchain's state by changing account information such as the funds. Since for unstructured data, there is no better way to store it than to keep all information separately, it would be highly inefficient to store all data directly inside the blockchain.

However, the data that is validated inside the blockchain has to be retrievable in some manner in order to make the concept of data transactions useful. To keep the load on the *validators* as low as possible, and to minimize the traffic within the system, it was decided to only store data from data transactions inside the *committee*. After all, the data stored inside the data transactions doesn't carry any information which is of immediate interest to any other *validators* in BAZO.

Since the amount of shards and therefore also *validators* in the blockchain can change over time, and the shard assignment changes after each epoch, it's not practical to distribute the responsibility of storing the data anyways. In order to keep a global view of the data, all nodes would have to be updated after each block height, which increases traffic in the blockchain and the workload of the *validators*, or the number of shards and the validator assignment would have to be kept stable, which contradicts the basic design principles of the sharded version of BAZO [11].

6.3.1 Data Summary

Whenever a data transaction gets validated by a *validator*, the *committee* takes the data from the transaction and stores it in the persistent database. For each sender account, the *committee* maintains a data structure in the database called data summary. The data summary consists of the following fields:

Address The sender's address of the data

Data A slice of bytestreams. It has the form `[]byte` and stores the data that the sender sent to the network

6.3.2 Storing Mechanism

Whenever a block which contains data transactions is validated at the committee, the function UpdateSummary(DataTx) is called, which is shown in Algorithm 9.

Algorithm 9: Updating the Data Summary:

UpdateDataSummary([]DataTx dataTx)

Input : []DataTx dataTx

Output: error err

```

[1] ...
[2] initialize updateMap
[3] initialize dataSummary
[4] ...
[5] forall dataTx in dataTx do
[6]   | updateMap[dataTx.From] = append(updateMap[dataTx.from], dataTx)
[7] end
[8] forall sender, dataTxSlice in updateMap do
[9]   | if  $\nexists$  dataSummary for sender in Database then
[10]     | create dataSummary
[11]     | forall dataTx in dataTxSlice do
[12]       | dataSummary.Data = append(dataSummary.Data, dataTx.Data)
[13]     | end
[14]     | ...
[15]     | write dataSummary to Database
[16]   | else
[17]     | fetch dataSummary for sender from Database
[18]     | forall dataTx in dataTxSlice do
[19]       | dataSummary.Data = append(dataSummary.Data, dataTx.Data)
[20]     | end
[21]     | write dataSummary to Database
[22]   | end
[23]   | ...
[24] end
[25] return nil

```

The argument *DataTx* contains all DataTx from the respective block. Since the data from the data transactions doesn't carry any semantics, the most sophisticated way how it can be stored in the data summary is by keeping a slice with all the data that has been validated, in addition with the address of the sender as a key to access the slice. Thus, a global and updated view of the data in the blockchain is kept at all time. UpdateSummary(DataTx) works in the following way: In order to be as efficient as possible, a map is created with the sender as a key and all of the sender's DataTx to be added in a slice as a the value. This map is successively filled up by iterating through the whole slice of *DataTx* to be added, which can be seen on Line 6. Then, the database

will be iterated by sender and not by transaction, minimizing the amount of database calls, demonstrated on Line 8. In each round, the database is checked if there is already a data summary for the sender. If yes, fetch it. If no, create a new data summary. Then, iterating through all DataTxns from the respective sender, the contained bytestreams in the individual DataTxns are appended to the slice in the data summary. After all DataTxns have been handled, the data summary is written back to the database.

In a later development phase, the committee could be queried by outsiders in order to retrieve information about the data from the blockchain. In order to achieve this, the committee has to be extended by an interface to allow for data summary requests.

6.4 Authentication & Validation

As it has been described in Section 5.8, there was a need for more rigorous security mechanisms. Therefore, in the committee version of BAZO, all messages have to be authenticated. The way how this is achieved is by using signatures. Moreover, some key messages will have to be checked more thoroughly, e.g. the epoch block and the state transitions. In the following Section, we will denote the authentication process as *verifying*, whereas the correctness check of the included data will be denoted as *validating*. The Sections describe how the previously described security measures are translated to an actual implementation in BAZO. To conclude the Section, there will be a presentation showing how these changes solved the issues which were previously present in the *sharded* version of BAZO.

6.4.1 Signing a Message

The first part of the new *verification* mechanism describes how the messages are signed by the senders.

Transactions

It has always been understood that transactions within BAZO have to be signed at least with the private key of the sender. Clients use an Elliptic Curve Digital Signature Algorithm to sign the hash of the transaction with the sender's private wallet key. Some transactions additionally require a signature by the private root wallet key.

Blocks

Blocks are signed using the private commitment key of the respective *validator*.

Epoch Blocks

Epoch blocks are signed using the private commitment key of the respective *validator*.

State Transitions

State transitions are signed using the private commitment key of the respective *validator*.

Transaction Assignments

Transaction assignments are signed using the private committee key of the respective committee member.

Committee Checks

Committee checks are signed using the private committee key of the respective committee member.

6.4.2 Verification of a Signature

The second part of the new *verification* mechanism describes how signed messages are verified by the receiver.

Transactions

Transactions are validated at the shards, before being added to a block. If the signature can be verified using the public wallet key of the sender, the transaction is also verified. Conversely, if the signature cannot be verified, the transaction cannot be verified either.

Blocks

Before a block is processed, it is checked if the block's beneficiary matches the validator shard assignment. That is, it is checked if the block has been produced by the *validator* who has been assigned to mine this exact block. In order to do that, the shard ID of the block is retrieved, and using the shard ID, the assigned *validator* is queried using the validator shard assignment. Then, using the public commitment key of the previously determined *validator*, the signature is checked. If the signature can be verified, then the block is also verified. Conversely, if the signature cannot be verified, the block cannot be verified either and the node keeps waiting for the right block to arrive. This check is performed inside the function `ValidateBlockSender(block)`, which returns a boolean depending if the check was successful or not.

Epoch Blocks

Before an epoch block is processed, it is checked if the epoch block's beneficiary matches the *leader* from the validator shard assignment. Then, using the public commitment key of the sender, the signature is checked. If the signature can be verified, then the epoch block is also verified. Conversely, if the signature cannot be verified, the epoch block cannot be verified either and the node keeps waiting for the right epoch block to arrive. This check is performed inside the function ValidateEpochBlockSender(*epochBlock*), which returns a boolean depending whether the check was successful or not.

State Transitions

Whenever a state transition is about to be processed, it is verified first. That means, that from the epoch block, the validator shard assignment is read. Using the validator shard assignment and the shard ID given in the state transition, it can be checked which validator the state transition originates from. Using the public commitment key of the sender, the signature is then checked. If the signature can be verified, then the state transition is also verified. Conversely, if the signature cannot be verified, the state transition cannot be verified either and the node keeps waiting for the right state transition to arrive. This check is performed inside the function ValidateStateTransition(*stateTransition*), which returns a boolean depending if the check was successful or not.

Transaction Assignments

Whenever a transaction assignment is about to be processed, it is verified first. That means, that from the epoch block, the committee leader's public committee key is read. Using this key, the signature will be tried to be verified. If the signature can be verified, then the transaction assignment is also verified. Conversely, if the signature cannot be verified, the transaction assignment cannot be verified either and the node keeps waiting for the right transaction assignment to arrive. This check is performed inside the function ValidateTransactionAssignment(*transactionAssignment*), which returns a boolean depending if the check was successful or not.

Committee Checks

Whenever a committee check is about to be processed, it is verified first. This means that using the address of the sender, the sender's public committee key is read. Using this key, the signature will be tried to be verified. If the signature can be verified, then the committee check is also verified. Conversely, if the signature cannot be verified, the committee check cannot be verified either and the node keeps waiting for the right committee check to arrive. This check is performed inside the function ValidateCommitteeCheck(*committeeCheck*), which returns a boolean depending if the check was successful or not.

6.4.3 Validation of a Message

As it has been mentioned before, certain messages need to be validated with special care. In particular, this Section will discuss how the validation flaws of the sharded version of BAZO were overcome. Exactly those messages which have the power to change the global state of the system need to be validated. Other than verification also happening inside the shards, all validation processes are only handled by the committee, except transactions, which are validated by the shards.

Note that in all cases the validation process is only executed after the verification process in order not to waste any resources.

Validation of Blocks

Blocks are validated by first checking the Proof-of-Stake condition. Afterwards it is checked whether the block contains any illegally added transactions by the block creator by comparing the transactions in the block to the transactions from the transaction assignment. If the transaction can't be found in the transaction assignment, the block creator is slashed. Finally it is checked whether the block contains any closed transactions. If this is the case, the transaction is again compared against the transaction assignment. If the transaction is in the transaction assignment, the *committee leader* is slashed. If it isn't, the block creator is slashed. If the Proof-of-Stake is valid and all of the following checks are negative, then block can be successfully validated. If the Proof-of-Stake is invalid, the block creator is slashed.

Validation of State Transitions

State transitions are validated in order to check whether the transactions which were validated in the corresponding block lead to the same relative state as the relative state which is present in the actual block. Therefore, upon reception of a block, its relative state is reconstructed using the function ReconstructRelativeState(*block*, *transactions...*). Inside the function, all transactions are processed one by one, step by step reaching the desired relative state. The mechanism not only considers funds flow, but also transaction fees and the block reward. After all relative states have been reconstructed and all state transitions have been received, the state transitions can be validated one-by-one using a simple for loop and a new function called compareRelativeStates(*relativeStateA*, *relativeStateB*) which is able to compare relative states. In order to keep things simple, only fund changes in the relative state are checked. If the reconstructed relative state matches the relative state from the state transition, the state transition is valid. Otherwise, the state transition's creator is slashed. This check can be informally described as follows:

$$RelativeState_{StateTransition} \stackrel{?}{=} \sum_{tzsInBlock} relativeStateChangeOfTransaction$$

Validation of Epoch Blocks

Epoch blocks are first validated by checking the Proof-of-Stake condition. Afterwards, the relative states from all of the received blocks are aggregated using the function `AggregateRelativeState(RelativeStateA, RelativeStateB)`. The return value of this function is the aggregated relative change of both relative states passed to the function as arguments, meaning that using a for loop and iteratively calling the function, the aggregated relative state from all blocks in the epoch can be reconstructed. Then the relative state between the previous epoch $n - 1$ and the current epoch n is compared to the aggregation of all calculated relative states from the blocks in the current epoch n using the function `compareRelativeStates(relativeStateA, relativeStateB)` which can be described informally by the following equation:

$$EpochState_n - EpochState_{n-1} \stackrel{?}{=} \sum_{shardIDs} block_{relativeState}$$

If both sides of the equation are the same, and the Proof-of-Stake is valid, the epoch block is valid. Otherwise, the epoch block creator, i.e. the *shard leader* is slashed.

Validation of Transaction Assignments

The scenario where a transaction assignment would be *invalid* is when it contains already closed transactions. Since only the committee keeps track of the global transaction pool, the validation of transaction assignments also exclusively happens there, which will be described as follows. Whenever a committee validates a block from the network, it checks whether a contained transaction is in the closed transactions pool already. If this is the case, an additional check finds out if this transaction was part of the transaction assignment. If this is the case, the transaction assignment is invalid and its creator, i.e. the committee leader is slashed. If no such transaction can be found, the transaction assignment is valid.

Validation of Committee Checks

Committee checks are essentially a summary of the validation steps which were performed by a particular committee during a particular round. In order not to leave the *committee leader* with complete power over the validation process, the validation results are evaluated against each other in the previously described byzantine consensus mechanism. In other words, the validation of the committee checks happens inside the byzantine consensus mechanism and is dependent of the information contained in the committee checks produced by other *committee members* in the system.

Unfortunately, however, this mechanism is not as rigorous as the one designed for other messages, because the *committee leader* could theoretically slash innocent nodes, or spare malicious nodes respectively, because it is the only node performing the byzantine consensus mechanism. A first effort to minimize the the effects of this vulnerability is to not

give the *committee leader* any incentives to slash innocent nodes, and to let the *committee leader* of epoch height n run the byzantine consensus mechanism of epoch height $n - 1$. This means, that a malicious *committee leader* cannot avoid punishments by sparing itself in the byzantine consensus mechanism. However, in future iterations of BAZO, this problem shall be addressed in more depth.

6.4.4 Security Issues in the Sharded Version of BAZO

This Section discusses the weaknesses that the previous implementation of the Sharding mechanism on BAZO showed, and for each of the weaknesses, explains how it was addressed in the *committee version* of BAZO.

Figure 6.5 describes a simplified flowchart of the consensus mechanism of the *sharded* version of BAZO. The separate steps are marked with a red number. In fact, at each one of these steps, malicious behaviour can occur, which is not prevented by the *sharded* version of BAZO.

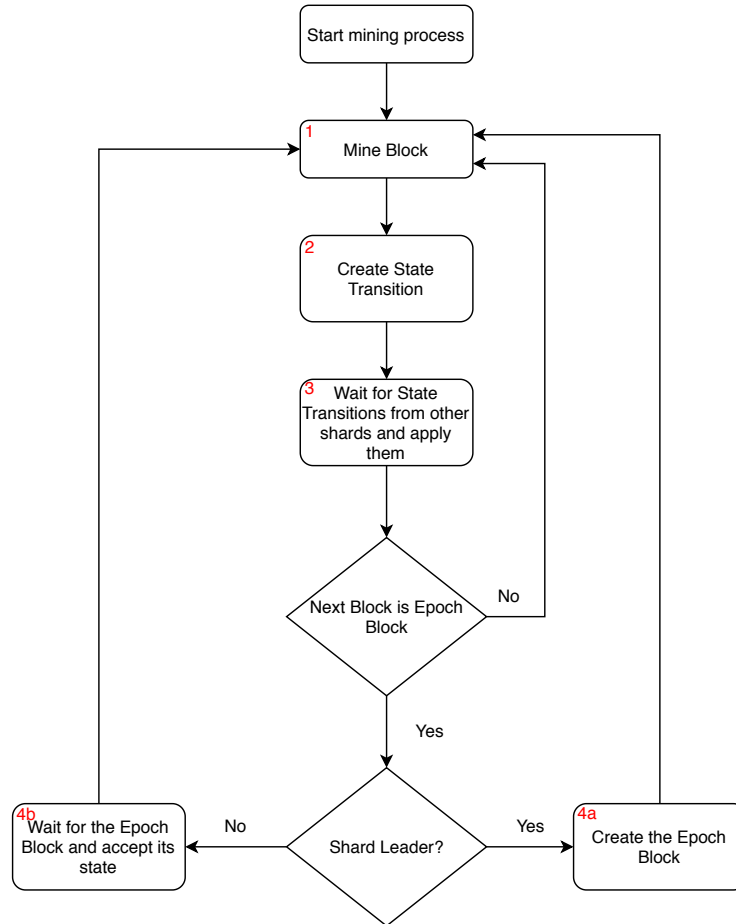


Figure 6.5: Flowchart of the previous Consensus Mechanism

Step 1 During block creation, invalid transactions, or already closed transactions could be put inside the block. In the proposed multi-validator per shard version by Kürsat

Aydinli [11], however, this issue could be resolved by letting other nodes in the shard validate the block as well. Moreover, *validators* could distribute blocks that they were not assigned to mine. This stays undetected, because the signature was only checked according to the block's creator, but it wasn't checked if the creator was actually assigned to mine that specific block. **Proposed Solution:** The committee validates the block, checking for falsely added transactions. Moreover, it authenticates the block based on the validator shard assignment.

Step 2 + 3 During state transition creation, any false information could be put into the relative state. Also, anyone could create a state transition under a wrong identity. **Proposed Solution:** There is a signature for the state transitions now. Every node in the system checks the signature of the state transition based on the validator shard assignment, making attacks much more difficult. Additionally, the *committee* validates the state transitions.

Step 4 There is no mechanism to perform authentication steps for the epoch block, which makes attacks possible. Moreover, the epoch state is not validated either. **Proposed Solution:** The committee checks the signature, PoS and authenticity of the epoch block based on the validator shard assignment, making attacks on authenticity and 51% attacks more difficult. Also, the state inside the epoch block is validated.

It can therefore be concluded, that the newly presented *committee* version helps to make the system more trustworthy and secure, which is an important step for the future development of BAZO.

6.5 New State Design

In order to accommodate the new entity set in the blockchain, i.e. the *committee members*, it was necessary to expand the state of the system by some fields. Note that analogously to the validators, each committee member is assigned to an RSA key pair, in this case denoted as the committee key. The added global fields are described below:

IsCommittee A boolean flag. If it's set to true, the connected account is a *committee member*. If it's set to false, the connected account is not a committee member.

CommitteeKey The public committee key of the connected account. Used for authentication.

6.6 New Communication Mechanisms

In order for the *committee version* of BAZO with its new functionalities to work, several communication mechanisms had to be newly implemented or changed. Their technical implementation details will be discussed in this Section.

6.6.1 State Transition

State transitions for the most part still serve the same purpose as they did in the sharded version of BAZO [11]: Helping to synchronize the global state. Due to the new transaction management system, however, it's not necessary anymore to add the hashes of transactions which other *validators* shall delete. However, authentication is necessary. Therefore, the issuer of the state transition needs to sign it with its private commitment key. The following fields make up a state transition:

ShardID The shard ID of the validator who issued the state transition

Height The height where the state transition was created

Signature Used for authentication

Relative State Change A map keeping the *relative* changes for each account in the global state

6.6.2 Transaction Assignment

For each *validator* in the network, and for each epoch height, a transaction assignment is created. The transaction assignments are all created by the committee *leader* and distributed to the shards and the rest of the committee alike. Transaction assignments can be used to see which *validator* is assigned to validate which transactions at a given height. Since *validators* don't permanently store any transactions, transaction assignments carry complete transactions instead of just the hashes. Moreover, the transaction assignment has to be signed by the issuer using the private committee key for authentication. The following fields make up a transaction assignment:

Shard ID The shard ID of the *validator* who gets assigned the transactions inside the transaction assignment

Height The height where the transaction assignment is to be used

Signature Used for authentication

Slices of AccTx, StakeTx, CommitteeTx, DataTx, FundsTx and FineTx One slice for each because the serializer only works with concrete classes

6.6.3 Committee Check

For each *committee member* in the network, and for each epoch height, a committee check is created. It contains information about what nodes the respective *committee member* votes to slash. It has to be signed using the private committee key of the issuer. The following fields make up a committee check:

Address The Address of the *committee member* who produced the committee check at hand

Height The height where the committee check was produced

Signature Used for authentication

Slices of committee member and validator addresses Slices containing information of what nodes should be slashed. Kept separately for easier handling at the byzantine consensus mechanism

6.7 New Transactions

In order for the committee version of BAZO to run and to fulfill its use case, some new transactions had to be added to the system. This Section covers their technical details.

6.7.1 CommitteeTx

For the committee version of BAZO, this new transaction is of very high importance, as it is used to add *committee members* to the system. It is initiated by the client interface and creates a new committee account in the state of the blockchain. In order to do that, a wallet key-pair and a committee RSA key-pair have to be created. Moreover, the transaction has to be signed with the private wallet key of the root node. The following parameters make up a CommitteeTx:

Wallet The public key of the wallet of the newly added committee *member*

Committee The public committee key of the newly added committee *member*

Fee The fee that has to be paid to the validator that validates the transaction

Signature Used for authentication

6.7.2 DataTx

For the committee version of BAZO, DataTx might make up the majority of transactions in the network, due to its IoT approach. They contain streams of data, which may come in a multitude of different formats. In order to keep a unified structure, however, it was decided to only let the sender of such transactions use bytestreams to add data to the transaction. Different formats may be used, but they have to be converted to bytestreams before sending and then reassembled when the data is retrieved. For authentication, it has to be signed by the sender's private wallet key. Moreover, it has to be additionally signed with the private wallet key of the root node. The following parameters make up a DataTx:

Sender The hash of the sender's address

Receiver The hash of the receiver's address

Data The data included in the DataTx. It has no practical size limit and is of the form *//byte*

Fee The fee that has to be paid to the validator that validates the transaction

Timestamp The number of nanoseconds elapsed since January 1, 1970 UTC

Signature Used for authentication

6.7.3 AggDataTx

The aggregation format and mechanisms are very similar to the Transaction Aggreagation of funds transactions in the merged version of BAZO [27]. If data transactions can be aggregated in a block, not the data transactions themselves, but the AggDataTx will be put into the block. AggDataTx's are not issued by the clients, but by the *validators* directly. Since during *verification* and *validation*, all data transactions inside the AggDataTx are checked separately, no additional mechanisms such as signatures are necessary. The following parameters make up an AggDataTx:

Sender The hash of the sender of the data transactions within the AggDataTx

Receivers The hashes of the receivers of the data transactions within the AggDataTx

AggregatedDataTxSlice The hashes of the data transactions included in the AggDataTx

6.7.4 FineTx

The FineTx can not be created by the client application. It is created by the committee as part of the byzantine consensus mechanism, and fines a specified user in the network by a certain amount. For authentication, it is signed with the sender's private wallet key. Overall, its structure is similar to a FundsTx. The following parameters make up a FineTx:

Sender The hash of the sender's address

Receiver The hash of the receiver's address. This is the address to be slashed

Fee The fee that has to be paid to the validator that validates the transaction

Amount The amount of funds to be deducted from the receiver's wallet

Signature Used for authentication

6.8 Timestamps

Since the main application of the new system is to support large streams of IoT Data, it was decided to remove the burden of having to add a Tx Count for the clients. In order to still allow for transactions to be properly hashed, especially to keep their uniqueness, a timestamp field was added to all transactions that previously required a Tx Count. It contains the number of nanoseconds elapsed since January 1, 1970 UTC. This timestamp does not have to be filled by the client itself, but it is automatically added to the transaction during construction. Such an implementation has the benefit of almost certainly guaranteeing the uniqueness of each timestamp, even if the internal clock is reset. This uniqueness is not given if the transaction counter is lost or reset.

Chapter 7

Evaluation

This Chapter discusses the performance testing results which were obtained from testing the system which was proposed in Chapter 5 and implemented in Chapter 6.

7.1 Theoretical Performance Evaluation

In theory, if the *committee* doesn't bottleneck the system, the performance could grow linearly according to the amount of *validators* added to the system. However, since the workload capacity of the *committee* is limited, and it's not scalable by adding more *committee members*, adding more *validators* will not in all cases increase the scalability of the system.

7.2 Local Testbed

First of all, the implementation at hand was again tested on a local testbed. A local testbed is easy to set up and shows what kind of performance can be reached in an almost ideal test scenario.

7.2.1 Implementation

For this purpose, more unit tests inside `..\miner\sharding_test.go` were written. In order to improve the TPS_{sent} , goroutines were used in a more sophisticated way this time. In Section 4.1.4, it was concluded that spawning one goroutine per artificial wallet lead to almost half of the sent transactions completely lost. Therefore, just one goroutine per *validator* was created, leading to rates of TPS_{sent} which were not high enough. Now, due to network and database improvements, a new mechanism is possible.

In the new local test, the idea is that the set of artificial wallets is partitioned into 8 partitions, each partition holding roughly the same amount of artificial wallets. For each

partition, a goroutine will then be spawned, sending out funds transactions from the artificial wallets in the partition to the root wallet. Inside each goroutine, of course, every *committee* member's port is addressed separately such that all *committee* members have the same complete transaction pool. Note that this implementation is different from Section 4.1.4, as now each goroutine sends the transactions to all *committee* members' ports, whereas before each *validator* port had its own goroutine. Partitioning the artificial wallet pool into more partitions than 8 didn't show any significant speed improvements anymore.

The setup of the testcase works in a similar way as in Section 4.1.4. At the beginning of the testrun, the artificial wallets are created. After they have been created, they are filled up with sufficient funds. After the setup phase ended, the goroutines are spawned and the benchmark starts.

In order to make the testcase more realistic, this time 60 artificial wallets were created, each sending out 3000 transactions, resulting in a total of 180'000 transactions. Just like in Section 4.1.6, the block size was kept at 800 Bytes and the transactions were sent in batches, making sure that there are transactions from as many different virtual wallets in the open memory pool as possible. The variables *numberOfRounds* and *txPerRound* were kept the same as in Section 4.1.4.

7.2.2 One Committee Member

The first testruns were conducted only using one committee member. This of course minimizes the overhead which will be experienced at the committee, but it will also provide the least rigorous security and trust. Still, the one committee case will provide the best example of what the new mechanism will be capable of when it comes to increasing the scalability by adding more *validators* to the network. That means, it will produce the highest possible TPS that the system can produce.

1 Committee Member - 1 Shard

This is the most simple implementation, and its interpretation resembles 1 shard - 1 validator scenario from Section 4.1.6. This means, that given 1 committee member, it defines the base TPS speed without *validators* working in parallel. The testing results can be taken from Table 7.1.

Table 7.1: *TPS* with 1 *Committee Member* and 1 *Validator* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	4285.7	873.8

Despite of all the added validation and security mechanisms, a solid improvement in TPS can be recorded compared to the results of the merged version of BAZO in Section 4.1.6.

1 Committee Member - 2 Shards

The results of the implementation with 2 shards can be retrieved from Table 7.2.

Table 7.2: *TPS* with 1 *Committee Member* and 2 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	3272.7	1353.4

It can be seen that the TPS almost doubles, which means that BAZO almost scales linearly by adding an additional *validator* to the system. Ideally, this trend would continue indefinitely.

1 Committee Member - 3 Shards

The results of the implementation with 3 shards can be retrieved from Table 7.2.

Table 7.3: *TPS* with 1 *Committee Member* and 3 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	3157.3	1698.1

It can be observed, that the benefit of adding an additional *validator* to the system was less significant this time. This already gives a hint that the *committee* might be reaching the limits of its capacity.

1 Committee Member - 4 Shards

The results of the implementation with 4 shards can be retrieved from Table 7.2.

Table 7.4: *TPS* with 1 *Committee Member* and 4 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	3000.0	1714.3

As it can be seen, the performance improvement of adding a 4th shard is negligible. This means, that for one committee member, the performance limit is reached already with 3 *validators*, while adding additional won't improve the TPS speed, because the *committee* is already working at full load.

7.2.3 Two Committee Members

In this test case, there are 2 committee members coordinating and validating the work the shards. Obviously, there is a lot more synchronization overhead than in the 1 committee version, so TPS drops are to be expected. Moreover, it could be observed that the TPS_{sent} dropped by almost half, as now two *committee members* have to be sent transactions to from each goroutine and the resources have to be shared. This behaviour has already been observed by Kürsat Aydinli in his master thesis in a very similar test case, where one goroutine was in charge of supplying multiple ports with transactions [11].

2 Committee Members - 1 Shard

From the test results with 2 *committee members* and 1 *shard*, it can be seen that the TPS drop isn't that drastic compared to the 1 committee version. The results can be found in Table 7.5.

Table 7.5: *TPS* with 2 *Committee Members* and 1 *Validator* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS_{sent}	TPS
800	180'000	180'000	1764.7	759.5

2 Committee Members - 2 Shards

The results of the implementation with 2 shards can be retrieved from Table 7.6.

Table 7.6: *TPS* with 2 *Committee Members* and 2 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS_{sent}	TPS
800	180'000	180'000	1730.8	1139.2

The improvements from adding one *validator* to the network is comparable to the results from the 1 committee version, although the absolute value is considerably lower in this configuration.

2 Committee Members - 3 Shards

The results of the implementation with 3 shards can be retrieved from Table 7.7.

Table 7.7: *TPS* with 2 *Committee Members* and 3 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS_{sent}	TPS
800	180'000	180'000	1800.0	1285.7

Unfortunately, it seems that the 2 *committee members* version is approximating its maximum capacity already, as the performance gain is only marginal compared to the 2 shards configuration.

2 Committee Members - 4 Shards

The results of the implementation with 4 shards can be retrieved from Table 7.8.

Table 7.8: *TPS* with 2 *Committee Members* and 4 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	1565.2	1084.3

Again, it can be seen that the TPS even decreased compared to the 3 shards configuration. In order to explain this behaviour, further research on BAZO has to be performed, but it's likely that the drop is due to an overload at the *committee*, which causes the whole system to slow down. The consequence of this would be that for each amount of *committee members* in the network, an ideal number of shards has to be found in order not to overload the *committee*.

7.2.4 Three Committee Members

As it was observed in the previous parts, there is a considerable drop in performance if a committee member is added. In order to find out if this trend continues, in this Section, a setup with three committee members will be evaluated. Note that the TPS_{sent} further decreased.

3 Committee Members - 1 Shard

The results of the implementation with 1 shard can be retrieved from Table 7.9.

Table 7.9: *TPS* with 3 *Committee Members* and 1 *Validator* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	1276.6	636.0

3 Committee Members - 2 Shards

The results of the implementation with 2 shards can be retrieved from Table 7.10.

Table 7.10: *TPS* with 3 *Committee Members* and 2 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	1295.0	861.2

3 Committee Members - 3 Shards

The results of the implementation with 3 shards can be retrieved from Table 7.11.

Table 7.11: *TPS* with 3 *Committee Members* and 3 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	1250.0	865.4

3 Committee Members - 4 Shards

The results of the implementation with 4 shards can be retrieved from Table 7.12.

Table 7.12: *TPS* with 3 *Committee Members* and 4 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	1139.2	810.8

The results from the implementation with 3 committee members again confirm the findings that were made before. The committee holds back the performance of the whole blockchain, and as soon as a certain threshold is overstepped, adding more shards only makes the system slower.

7.2.5 Four Committee Members

To conclude this Section, a test with 4 committee members was performed as well. Another substantial drop in TPS_{sent} was observed.

4 Committee Members - 1 Shard

The results of the implementation with 1 shard can be retrieved from Table 7.13.

Table 7.13: *TPS* with 4 *Committee Members* and 1 *Validator* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	878.0	433.7

4 Committee Members - 2 Shards

The results of the implementation with 2 shards can be retrieved from Table 7.14.

Table 7.14: *TPS* with 4 *Committee Members* and 2 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	1034.5	743.8

4 Committee Members - 3 Shards

The results of the implementation with 3 shards can be retrieved from Table 7.15.

Table 7.15: *TPS* with 4 *Committee Members* and 3 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	886.6	750.0

4 Committee Members - 4 Shards

The results of the implementation with 4 shards can be retrieved from Table 7.16.

Table 7.16: *TPS* with 4 *Committee Members* and 4 *Validators* in the Committee Version

Block Size	Transactions			
Bytes	#sent	#validated	TPS _{sent}	TPS
800	180'000	180'000	918.4	684.4

It can be seen from the results that in the 4 *committee members* version, the TPS gets close to the TPS_{sent}. This means that the TPS_{sent} will again bottleneck the testing until a better testbed is found. For this reason, no testings with more than 4 *committee members* are performed.

Other than that, the results confirm those findings from the previous test runs.

7.2.6 Testing Conclusions

Figure 7.1 summarizes the testing results. It can be seen that the test results with different amounts of *committee members* almost behave in the same way, but they are shifted on the Transactions Per Second axis. Furthermore, it appears that the slope becomes less steep if there are more *committee members*, which means that smaller *committees* have a higher scaling potential.

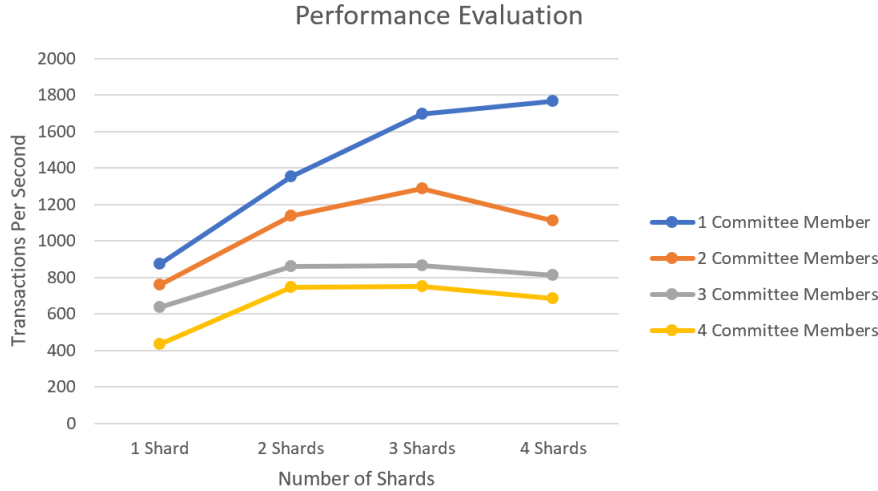


Figure 7.1: Evaluation with a varying Amount of Committee Members and Validators

Also, it can be observed that the Sharding mechanism now has the potential to give a considerable scalability benefit to BAZO, as opposed to the Merged version of BAZO, which was tested in Section 4.1.6. This can mainly be explained by the enhanced *transaction management*, because now the most time-intense work is better distributed among the shards. However, once a certain TPS threshold is reached, this threshold decreasing with a growing committee, adding more miners doesn't improve the scalability anymore, at some point even slowing down the system. It is evident that the reason for this is the committee, which at a certain point reaches its maximum capacity.

It follows that there is a trade-off between scalability and security in the current system. If the system shall validate transactions as fast as possible, then a small committee might be a good option. Alternatively, if security concerns prevail, a larger committee would be advisable. Judging by the testing, a 2 committee members - 3 shards configuration might be a good choice, since it doesn't completely centralize the committee while benefiting from the performance gains that the Sharding mechanism provides.

In order to get more insights to the performance of BAZO, in future test runs more metrics could be included. However, for the scope of this thesis, evaluating the performance mainly based on the TPS is sufficient as it is by far the most interesting measure of scalability. Moreover, the TPS_{sent} has to be further increased in order to be able to test builds with even more active nodes.

7.3 Online Testbed

Unfortunately, at the end of the thesis work, there is no online testbed for BAZO which supplies any meaningful data. This, however, does not imply that no work was done to further investigate the problems and limits of BAZO's p2p structure. In this Section, an implementation of a global BAZO testbed will be proposed. Afterwards, the limitations of the current implementation of BAZO will be shown along with a description of where the problem lies.

7.3.1 Implementation

The global testbed runs on a network provided by GCP(Google Cloud Platform). One node was placed in Iowa, one was placed in South Carolina, one in Osaka and one in Mumbai. For the various tests performed, the entity assignments continuously changed. Since the latency isn't significantly influenced by the distance between two nodes [27], this did not in any way influence the tests. Instructions on how to install BAZO on the GCP infrastructure can be taken from Fabio Maddaloni's thesis [27].

7.3.2 Current Problems

Unfortunately, deploying the current implementation of BAZO online yielded unforeseeable problems: When running the system, the communication between two nodes would often stop, without a crash of the connection being reported to the user, i.e. the reader of the log files and the command line output. The reason for this behaviour is underlying architecture of how two nodes communicate with each other in BAZO. From a fixed node n , for each known other node o in the network, a goroutine called `go peerConn( $p$  *peer)` is spawned. Inside this goroutine, n keeps listening for messages from o . For each incoming message, it is read from the network and assembled byte by byte inside the function `RcvData( $p$  *peer)`. Now, if the possible message length is increased from the previously set value of 80'000 Bytes, which it has to in order to create sufficiently large transaction assignments, it happens very often that a `peerConn( $p$  *peer)` goroutine gets stuck inside the `RcvData( $p$  *peer)` function. Only after its execution is finished can n listen to messages from o again. Note that this only impacts traffic in one direction. This means that it is possible that n requests something from o , and o sends the response. However, the response doesn't reach n , since its goroutine for o is busy. Such an error can take several hours to recover, blocking the whole blockchain in the process.

In a nutshell, this means that in order to support the idea behind the *committee version* of BAZO, the messages need to be able to take on a large bytesize. However, if the message bytesize is increased, the network becomes unreliable. If the bytesize is kept at 80'000 Bytes, the online implementation works, but no meaningful load tests can be performed.

To fix this problem, extensive research has to be done on how the peer network packages work in BAZO, and maybe even a better alternative has to be found. Also, alternatives on how messages can be received from the network have to be evaluated.

7.4 Future Work

If more work is to be performed on the *committee version* of BAZO, several issues should be addressed. Those will be mentioned in the following Section.

Firstly, in order to prove that the implementation can perform convincingly on a distributed system, the concerns from Section 7.3 have to be fixed. This is crucial, however it will require expertise with network systems and structures.

Moreover, in order to make full use of the transaction pool partitioning mechanism, a better way to partition transactions should be found. At the moment, it is possible that one shard has a much higher workload than another shard at a certain block height, which leads to delays which propagate over the whole blockchain. An extreme example would be a scenario where all transactions in the system have the same sender - adding more *shards* would in this scenario not increase the scalability in any way, because the transaction assignment mechanism is purely based on the sender of the transaction.

A more difficult point which should be addressed is the performance of the committee. As it became clear from the testing in Section 7.1, the committee starts bottle-necking the shards as soon as it reached its capacity limit. Possible solution approaches would be to further optimize the performance of the committee by code parallelization, or by distributing the work of the individual committee members more instead of replicating every step at every committee member. In short, it should be possible to scale the performance of the committee, just like it is the case with the shards.

As already mentioned in Section 6.3, the committee could be extended to allow queries for data transactions from outside.

Finally, as it is clear from Sections 6.1 and 6.4, the consensus mechanism is not perfect yet, especially because all malicious behaviour is punished in the same way, and the *committee leader* is left with too much power. Some time has to be spent evaluating methods on how malicious nodes can be punished in a reasonable way in order for the blockchain to keep its integrity and to deter possible attacks from happening.

Chapter 8

Summary & Conclusions

As it became clear in the previous chapters, BAZO evolved a lot in the course of this thesis. While roughly the first half of the work was dedicated to fixing existing bugs and merging the projects of Transaction Aggregation [27] and Sharding [11], the second half was dedicated to increasing the scalability and security of the overall system.

The clearly more interesting part is the scalability improvement. From the beginning of the thesis, it was clear that this work would be very experimental in its nature, which can be seen best in Chapter 4. For instance, a new design was proposed and implemented in Section 4.1.2, only to be dropped after its evaluation because it didn't improve the system. However, significant advancements were made in Section 4.2, mostly due to database tweaks. Those improvements, in fact, increased the TPS by a considerable margin already, such that it was decided to not only invest time to further enhance the scalability, but also to tighten the security of the system, which was only very rudimentary after the Sharding mechanism had been introduced [11].

The solution that was presented to address both scalability and security issues in Chapter 5 was influenced by cutting-edge contemporary research and developments in the blockchain community as it has been evaluated in Section 2.3, while taking the underlying constraints of BAZO into consideration. The proposed design is then compared to the aforementioned research items in Section 5.9, implemented in Chapter 6 and evaluated in Chapter 7.

The Evaluation shows that it is now indeed possible to scale the TPS of BAZO almost linearly by adding additional nodes, until the committee starts bottlenecking the blockchain. This is an expected behaviour, since the workload is not partitioned inside the committee as it is in the shards. Therefore, it can be concluded that the work of this thesis successfully enables BAZO to scale with adding more *validators*, at least up to a certain point. However, the new *committee* which had to be created to support this mechanism in turn slows down the system again. Thus, while one problem was solved, a new one was created. Still, this thesis shall serve as an inspiration for further iterations of BAZO, where a version of the *committee* could be implemented which does not bottleneck the *shards*, and it gives some guidance for techniques that can be leveraged to scale a blockchain. Despite the drawbacks of the design and implementation, the aspired transaction validation rate of 1000-2000 TPS which has been defined as a goal for this thesis has been reached.

However, arguably the main weakness of this thesis is how the implementations were tested. The bugfixing, implementations and also the preparations on GCP were immensely time-consuming, which meant that in the end, not enough time was left to get results from an online testbed. This was mainly caused by the p2p package of BAZO, which lead to problems throughout the thesis, and in several sections lead to behaviours which could not be explained with complete certainty, such as the fluctuations of TPS_{sent} and the corrupted messages.

Bibliography

- [1] Bitfinex: Zilliqa. <https://support.bitfinex.com/hc/en-us/articles/360007399554-Zilliqa>. Accessed: 20.02.2020.
- [2] Internet of Things forecast. <https://www.ericsson.com/en/mobility-report/internet-of-things-forecast>. Accessed: 09.02.2020.
- [3] Investopedia: Blockchain. <https://www.investopedia.com/terms/b/blockchain.asp>. Accessed: 12.03.2020.
- [4] Libra docs: Consensus. <https://developers.libra.org/docs/crates/consensus>. Accessed: 19.02.2020.
- [5] Libra whitepaper. https://libra.org/en-US/wp-content/uploads/sites/23/2019/06/LibraWhitePaper_en_US.pdf. Accessed: 26.02.2020.
- [6] QuarkChain Whitepaper. <https://quarkchain.io/wp-content/uploads/2018/11/QUARK-CHAIN-Public-Version-0.3.5.pdf>. Accessed: 26.02.2019.
- [7] Sharding Plus Cross-chain: How Will QuarkChain, A Public Chain Closely Related to Facebook, Win Out? <https://medium.com/quarkchain-official/sharding-plus-cross-chain-how-will-quarkchain-a-public-chain-closely-related-to-facebook-win-d05dedaac702>. Accessed: 19.02.2019.
- [8] Tendermint github repository. <https://github.com/tendermint/tendermint>. Accessed: 19.02.2020.
- [9] Which is more expensive? for loop or database call? <https://stackoverflow.com/questions/1390757/which-is-more-expensive-for-loop-or-database-call>. Accessed: 17.03.2020.
- [10] Zilliqa Is A Scalable, Secure Public Blockchain Platform. <https://zilliqa.com/>. Accessed: 29.12.2019.
- [11] Kursat Aydinli. Design and implementation of a scalable iot-based blockchain. <https://files.ifl.uzh.ch/CSG/staff/Rafati/Kursat-Aydinli-MA.pdf>. Accessed: 29.12.2019.
- [12] Simon Bachmann. Proof Of Stake For Bazo. <https://files.ifl.uzh.ch/CSG/staff/bocek/extern/theses/BA-Simon-Bachmann.pdf>. Accessed: 09.02.20120.

- [13] Matthew Beedham. Blockchain Sharding Made So Simple Your Dog Would Understand. <https://thenextweb.com/hardfork/2019/01/18/explainer-blockchain-sharding-beginners/>. Accessed: 29.12.2019.
- [14] Coin Crunch. QuarkChain Review — A New Scalable Blockchain Looking to Dethrone Ethereum? <https://hackernoon.com/quarkchain-review-a-new-scalable-blockchain-looking-to-dethrone-ethereum-9ffc0e814772>. Accessed: 19.02.2019.
- [15] Brian Curran. What is Sharding? Guide to this Ethereum Scaling Concept Explained. <https://blockonomi.com/sharding/>. Accessed: 09.02.2020.
- [16] Mark Drake. Understanding Database Sharding. <https://www.digitalocean.com/community/tutorials/understanding-database-sharding>. Accessed: 29.12.2019.
- [17] Ravi Duddukuru. Permissioned vs Permissionless Blockchain: Why did Libra start with Permissioned? https://medium.com/@ravi_97980/permissioned-vs-permissionless-blockchain-why-did-libra-start-with-permissioned-93eeb971189b. Accessed: 20.02.2019.
- [18] Jake Frankenfield. Permissioned blockchains. <https://www.investopedia.com/terms/p/permissioned-blockchains.asp>. Accessed: 19.02.2020.
- [19] Bennett Garner. What Is Zilliqa (ZIL)? | The Complete Guide to the High Throughput Blockchain. <https://coincentral.com/zilliqa-beginners-guide/>. Accessed: 01.01.2019.
- [20] HashSpike. Zilliqa: A game-changer for blockchain scalability? <https://hackernoon.com/zilliqa-a-game-changer-when-it-comes-to-blockchain-scalability-4fb1c13b1b8a>. Accessed: 16.03.2020.
- [21] Obasi Ifegwu. Binance academy: Finality. <https://www.binance.vision/glossary/finality>. Accessed: 20.02.2020.
- [22] Zac Johnson. The advantages of a faster blockchain for global and financial institutions. <https://www.influencive.com/advantages+of+a+faster+blockchain>. Accessed: 15.02.2020.
- [23] Eleftherios Kokoris Kogias, Philipp Svetolik Jovanovic, Linus Gasser, Nicolas Gailly, Ewa Syta, and Bryan Alexander Ford. Omniledger: A secure, scale-out, decentralized ledger via sharding. page 16, 2018.
- [24] Alok Kothari. Understanding Harmony’s Cuckoo Rule for Resharding. <https://medium.com/harmony-one/understanding-harmonys-cuckoo-rule-for-resharding-215766f4ca50>. Accessed: 19.02.2019.
- [25] Kenny Li. The Blockchain Scalability Problem the Race for Visa-Like Transaction Speed. <https://hackernoon.com/the-blockchain-scalability-problem-the-race-for-visa-like-transaction-speed-5cce48f9d44>. Accessed: 09.02.2020.

- [26] Jameson Lopp. Thoughts on libra “blockchain”. <https://blog.lopp.net/thoughts-on-libra--blockchain-/>. Accessed: 19.02.2020.
- [27] Fabio Maddaloni. Evaluation and improving scalability of the bazo blockchain. <https://files.ifi.uzh.ch/CSG/staff/Rafati/Fabio-Maddaloni-BA.pdf>. Accessed: 29.12.2019.
- [28] Mike Isaac Nathaniel Popper. How libra, facebook’s cryptocurrency, would work for you. <https://www.nytimes.com/2019/06/18/technology/how-libra-would-work-for-you.html>. Accessed: 19.02.2020.
- [29] Jeff S. Neo’s dbft 2.0 — single block finality with improved availability. <https://medium.com/neo-smart-economy/neos-dbft-2-0-single-block-finality-with-improved-availability-6a4aca7bd1c4>. Accessed: 19.02.2020.
- [30] Livio Sgier. Bazo – A Cryptocurrency From Scratch. <https://files.ifi.uzh.ch/CSG/staff/bocek/extern/theses/BA-Livio-Sgier.pdf>. Accessed: 09.02.2020.
- [31] Quark Chain Team. After Scaling, Why Do Public Chains Still Fail to Land? <https://medium.com/quarkchain-official/after-scaling-why-do-public-chains-still-fail-to-land-61eb74bdabe>. Accessed: 19.02.2019.
- [32] Bozhi Wang, Shiping Chen, Lina Yao, Bin Liu, Xiwei Xu, and Liming Zhu. *A Simulation Approach For Studying Behavior And Quality Of Blockchain Networks*, pages 18–31. Springer, Cham, 06 2018.
- [33] Mahdi Zamani, Mahnush Movahedi, and Mariana Raykova. Rapidchain. *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018.
- [34] Jiashuo Zhang, Jianbo Gao, Zhenhao Wu, Wentian Yan, Qize Wu, Qingshan Li, and Zhong Chen. Performance analysis of the libra blockchain: An experimental study, 2019.

Abbreviations

IoT	Internet of Things
PoS	Proof of Stake
TPS	Transactions per Second
BFT	Byzantine Fault Tolerance
Tx	Transaction
UTXO	Unspent Transaction Outputs
FundsTx	Funds Transaction
AggTx	Aggregated Funds Transaction
DataTx	Data Transaction
AggDataTx	Aggregated Data Transaction
AccTx	Account Transaction
StakeTx	Stake Transaction
FineTx	Fine Transaction
IPv4	Internet Protocol Version 4
IPv6	Internet Protocol Version 6
ECDSA	Elliptic Curve Digital Signature Algorithm
RSA	Rivest–Shamir–Adleman (cryptosystem)
json	JavaScript Object Notation
p2p	Peer-to-Peer
EVM	Ethereum Virtual Machine
I/O	Input/Output
RAM	Random-Access Memory
EPFL	École Polytechnique Fédérale de Lausanne
GCP	Google Cloud Platform

List of Figures

2.1	Concept of Transaction Aggregation	4
2.2	Concept of Sharding	5
4.1	Star and Mesh Network Topology	18
4.2	Multiple Nodes Per Shard Design	19
4.3	Flowchart of the new State Synchronization	23
4.4	Schematic Representation of the Transaction Management Overhead in the <i>Validators</i>	31
4.5	Process of writing closed Transactions to Database	34
4.6	Process of writing closed Transactions to Database using Batching	35
5.1	Entities in BAZO	42
5.2	Schematic Representation of the Validation Process	42
5.3	Schematic Representation of the Transaction Management from the Val- idator's Perspective	45
5.4	Schematic Representation of the Transaction Management from the Com- mittee's Perspective	45
5.5	Schematic Representation of the Communication Mechanism	48
6.1	Flowchart of the Shards	56
6.2	Flowchart of the Committee	58
6.3	Schematic Representation of the Transaction Rebroadcasting	62
6.4	Schematic Representation of the new Transaction Lifecycle	63
6.5	Flowchart of the previous Consensus Mechanism	71
7.1	Evaluation with a varying Amount of Committee Members and Validators	84

List of Tables

4.1	Table of different Block Sizes influencing the <i>TPS</i> with 1 <i>Validator</i>	30
4.2	<i>TPS</i> with 2 <i>Validators</i>	30
4.3	<i>TPS</i> with 2 <i>Validators</i> . Accelerated.	32
4.4	<i>TPS</i> with 3 <i>Validators</i> . Accelerated.	33
4.5	<i>TPS</i> with 3 <i>Validators</i>	33
4.6	<i>TPS</i> with 1 <i>Validator</i> in the improved Version	37
4.7	<i>TPS</i> with 2 <i>Validators</i> in the improved Version	37
7.1	<i>TPS</i> with 1 <i>Committee Member</i> and 1 <i>Validator</i> in the Committee Version	78
7.2	<i>TPS</i> with 1 <i>Committee Member</i> and 2 <i>Validators</i> in the Committee Version	79
7.3	<i>TPS</i> with 1 <i>Committee Member</i> and 3 <i>Validators</i> in the Committee Version	79
7.4	<i>TPS</i> with 1 <i>Committee Member</i> and 4 <i>Validators</i> in the Committee Version	79
7.5	<i>TPS</i> with 2 <i>Committee Members</i> and 1 <i>Validator</i> in the Committee Version	80
7.6	<i>TPS</i> with 2 <i>Committee Members</i> and 2 <i>Validators</i> in the Committee Version	80
7.7	<i>TPS</i> with 2 <i>Committee Members</i> and 3 <i>Validators</i> in the Committee Version	80
7.8	<i>TPS</i> with 2 <i>Committee Members</i> and 4 <i>Validators</i> in the Committee Version	81
7.9	<i>TPS</i> with 3 <i>Committee Members</i> and 1 <i>Validator</i> in the Committee Version	81
7.10	<i>TPS</i> with 3 <i>Committee Members</i> and 2 <i>Validators</i> in the Committee Version	82
7.11	<i>TPS</i> with 3 <i>Committee Members</i> and 3 <i>Validators</i> in the Committee Version	82
7.12	<i>TPS</i> with 3 <i>Committee Members</i> and 4 <i>Validators</i> in the Committee Version	82
7.13	<i>TPS</i> with 4 <i>Committee Members</i> and 1 <i>Validator</i> in the Committee Version	83
7.14	<i>TPS</i> with 4 <i>Committee Members</i> and 2 <i>Validators</i> in the Committee Version	83

7.15	<i>TPS</i> with 4 <i>Committee Members</i> and 3 <i>Validators</i> in the Committee Version	83
7.16	<i>TPS</i> with 4 <i>Committee Members</i> and 4 <i>Validators</i> in the Committee Version	83

Appendix A

Installation Guidelines

This chapter contains the installation guidelines to run BAZO.

A.1 Installation

Since BAZO's implementation changed drastically during the thesis, several changes on how the system is started had to be performed. However, most elements are taken from previous versions of BAZO [11] [27] [12] [30].

First of all, Go has to be installed. The implementation at hand has been tested on Go version 1.12.7. The latest build of Go can be fetched from the following website:

<https://golang.org/dl/>

After downloading and executing the install file, the *GOPATH* and *GOROOT* have to be set. The following website will provide some guidance:

<https://github.com/golang/go/wiki/SettingGOPATH>

Afterwards, the BAZO Miner and BAZO Client can be cloned and run. For the final committee version of BAZO, the following branches should be checked out:

Miner: github.com/oigele/bazo-miner/tree/ProjectForIoTMoreChecksAndMoreCommittees

Client: github.com/oigele/bazo-client/tree/ClientForIoT

For running the system, the *README.md* files in the respective repositories supply sufficient information. Additionally, in the *README.md* of the Miner application, there is a comprehensive tutorial on how to bootstrap a system consisting of 2 *validators* and 2 *committee members*. Moreover, all commands that can be used by the respective Applications are thoroughly explained there, including the commands which were added as part of this thesis.

For successful implementation, however, some points should be considered. Firstly, every wallet, commitment and committee file has to be used properly, otherwise there will

always be issues. If for example the wrong name is used, BAZO will just create a new file. However, since its keys aren't correct anymore, the system won't run properly. Moreover, for local use, never use localhost as the IP address. The BAZO p2p package is only capable of detecting IPv4 addresses, so the actual local IPv4 address of the machine should be given. In most cases, this should be *127.0.0.1*.

It might happen that there arise problems with the **cli** library used from *github.com/urfave/cli*. In that case, manually change the branch to *v1* instead of *master*. The reason for this is that some features which are required in BAZO have been deprecated as of December 2019.

A.2 Online Testbed

For guidance on how a global testbed can be implemented based on BAZO, it is advised to consult Fabio Maddaloni's thesis [27].

Appendix B

Contents of the CD

- Miner Application Source Code
- Client Application Source Code
- L^AT_EX Source Code
- Final thesis PDF
- Midterm Presentation & Final Presentation
- Relevant Log Files